

# mono3d节点设计文档

## 1. 总体设计

### 1.1 模块功能

 mono3d节点包含pv\_nn网络推理节点和pv\_post网络后处理节点两个子节点，两个节点进程解耦，pv\_post依赖pv\_nn输入

- Mono3D网络节点推理输出
- 针孔目标帧间目标关联
- 激光点云目标位置校正
- 目标属性优化

### 1.2 模块输入

#### 1.2.1 PV\_NN

| 输入消息名称                          | 消息格式                                             | 消息含义    |
|---------------------------------|--------------------------------------------------|---------|
| /ego_tf_rel                     | LocalizationEstimate                             | 自车位姿    |
| /camera_xxx/image_color<br>/yuv | sensor_msgs::Image<br>sensor_msgs::ImageLowDelay | YUV图像信息 |

#### 1.2.2 PV\_POST

| 输入消息名称                    | 消息格式                     | 消息含义              |
|---------------------------|--------------------------|-------------------|
| /ego_tf_rel               | LocalizationEstimate     | 自车位姿              |
| /pv_xxx_small             | ObjectInnerArray         | 2D框与网络输入图像匹配的检测结果 |
| /pv_xxx_gray              | sensor_msgs::Image       | 网络输入图像的灰度图        |
| /lidar_xxx/rslidar_points | sensor_msgs::PointCloud2 | 激光雷达点云            |

|                        |                          |                    |
|------------------------|--------------------------|--------------------|
| /lidar_xxx/range_image | sensor_msgs::RangeImage2 | 激光雷达RangeImage（可选） |
| /radar_xxx             | ObjectInnerArray         | 毫米波雷达信息（可选）        |

### 1.3 模块输出

#### 1.3.1 PV\_NN

| 输出消息名称         | 消息格式               | 消息含义              |
|----------------|--------------------|-------------------|
| /pv_xxx_object | ObjectInnerArray   | 2D框与原图匹配的检测结果     |
| /pv_xxx_gray   | sensor_msgs::Image | 网络输入图像的灰度图        |
| /pv_xxx_small  | ObjectInnerArray   | 2D框与网络输入图像匹配的检测结果 |

#### 1.3.2 PV\_POST

| 输出消息名称              | 消息格式             | 消息含义                 |
|---------------------|------------------|----------------------|
| /pv_post_xxx_object | ObjectInnerArray | 2D框与原图匹配的检测结果（用于可视化） |

| ObjectInnerArray.msg |          |                       |      |             |             |    |                                          |
|----------------------|----------|-----------------------|------|-------------|-------------|----|------------------------------------------|
| 一级字段                 | 二级字段     | 类型                    | 字节大小 | 取值范围        | 默认值         | 单位 | 含义                                       |
| header               | seq      | uint32                | 4    | [0, 2^32-1] | 0           | -  | sequence ID: consecutively increasing ID |
|                      | time     | ros::time or uint64_t | 8    | [0, 2^64-1] | 0           | ns | 时间                                       |
|                      | frame_id | string                | -    | -           | "base_link" | -  | 坐标系，默认是base_link自车坐标系                    |
| sensor_id            | -        | uint8                 | 1    | [0, 255]    | 0           | -  | 目标传感器序列号，具体见下方                           |
| object_list          | -        | vector<ObjectInner>   | -    | -           | -           | -  | 障碍物列表                                    |

|          |   |              |   |   |   |   |         |
|----------|---|--------------|---|---|---|---|---------|
| curb_set | - | vector<Curb> | - | - | - | - | 矢量化路沿集合 |
|----------|---|--------------|---|---|---|---|---------|

| ObjectInner.msg  |          |         |          |             |         |     |                                                                                                                                                             |
|------------------|----------|---------|----------|-------------|---------|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 一级字段             | 二级<br>字段 | 类型      | 字节<br>大小 | 取值范围        | 默认<br>值 | 单位  | 含义                                                                                                                                                          |
| object_id        | -        | uint32  | 4        | [1, 2^32-1] | 0       | -   | 目标序列号                                                                                                                                                       |
| type             | -        | uint8   | 1        | [0, 255]    | 0       | -   | 目标类别                                                                                                                                                        |
| type_confidence  | -        | float32 | 4        | [0, 1]      | 0       | -   | 目标类别置信度<br><br>RADAR目标:如果不存在表示60C丢失<br><br>0x0: 无效<br>0x1: <25%<br>0x2: <50%<br>0x3: <75%<br>0x4: <90%<br>0x5: <99%<br>0x6: <99.9%<br><u>0x7: &lt;=100%</u> |
| box_full         | -        | BBox2D  | 16       | -           | 0       | pix | 目标想象框                                                                                                                                                       |
| box_visible      | -        | BBox2D  | 16       | -           | 0       | pix | 目标可见框                                                                                                                                                       |
| left_door_box    | -        | BBox2D  | 16       | -           | 0       | pix | 目标左车门框                                                                                                                                                      |
| left_door_score  | -        | float32 | 4        | -           | 0       | -   | 目标左车门框分数                                                                                                                                                    |
| right_door_box   | -        | BBox2D  | 16       | -           | 0       | pix | 目标右车门框                                                                                                                                                      |
| right_door_score | -        | float32 | 4        | -           | 0       | -   | 目标右车门框分数                                                                                                                                                    |
| box_center_base  | -        | Vec3D   | 12       | [-500, 500] | 0       | m   | 自车坐标系下目标 3D box 中心点坐标                                                                                                                                       |
| yaw_base         | -        | float32 | 4        | (-pi, pi]   | 0       | rad | 自车坐标系下目标 3D box 车头朝向                                                                                                                                        |

|                   |        |         |   |                 |   |       |                                                                                                                                                                                                                                                                                                                                                |
|-------------------|--------|---------|---|-----------------|---|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| box_size          | length | float32 | 4 | [0, 100]        | 0 | m     | 目标 box 长度（朝向方向）                                                                                                                                                                                                                                                                                                                                |
|                   | width  |         |   | [0, 100]        | 0 | m     | 目标 box 宽度（垂直朝向方向）                                                                                                                                                                                                                                                                                                                              |
|                   | height |         |   | [0, 100]        | 0 | m     | 目标 box 高度（地面法线方向）                                                                                                                                                                                                                                                                                                                              |
| yaw_rate_base     | -      | float32 | 4 | [-3, 3]         | 0 | rad/s | 目标 box 的旋转角速度                                                                                                                                                                                                                                                                                                                                  |
| velocity_base     | -      | Vec3D   | 4 | [-250, 250]     | 0 | m/s   | 自车坐标系下目标 box 中心点绝对速度                                                                                                                                                                                                                                                                                                                           |
| acceleration_base | -      | Vec3D   | 4 | [-60, 60]       | 0 | m/s   | 自车坐标系下目标 box 中心点绝对加速度<br>pv_nn: <ul style="list-style-type: none"> <li>相机坐标系锚点直接计算的相对速度</li> </ul>                                                                                                                                                                                                                                             |
| motion_type       | -      | uint8   | 1 | {0, 1, 2, 3, 4} | 0 | -     | 0: Unknown -- default<br>1: Stationary -- static object like cone or ashcan<br>2: Stopped ----- the state from moving to stationary<br>3: MovingSlowly ---- the speed of moving object is below 5km/h<br>4: Moving ---- the speed of object is above 5km/h<br><br>bev_nn目标只输出(stationary和moving这两类)<br>Occupancy: Stationary代表静目标 Stopped代表动目标 |
| nn_state          | -      | uint8   | 1 | {0, 1}          | 0 | -     | 目标综合置信度<br>(NN用于挖数据)<br><br>0: 下游可用<br><br>1/2/3: 不同的过滤方式筛掉的目标，下游不可用，用于数据挖掘<br><br><a href="#">1. 低分</a>                                                                                                                                                                                                                                       |



|                    |   |       |   |                  |   |   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------|---|-------|---|------------------|---|---|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                    |   |       |   |                  |   |   | <div>2. 截断</div> <div>3. 遮挡</div> <div>pv_post计算nn_state用于表示目标状态</div> <div>0. 正常</div> <div>2. 截断</div> <div>3. 遮挡</div>                                                                                                                                                                                                                                                                                                                                                                                                           |
| lidar_camera_check | - | uint8 | 1 | 填写关联相机的sensor_id | 0 | - | <div>激光3D目标是否被视觉2D确认</div> <div>0: 没有被视觉2D确认</div> <div>其它相机的sensor_id的枚举值</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| is_box_refine      | - | uint8 | 1 | {0, 1, 2}        | 0 | - | <div>bev_nn:</div> <div>0: 正常目标</div> <div>2: fov外的框</div> <div>bev_post: 是否对box进行精修:</div> <div>0: 没有精修</div> <div>1: 对box精修</div> <div>2: fov外的框</div> <div>pv_nn</div> <div>0 网络速度不可用</div> <div>1 网络</div> <div>pv_post:</div> <div>对于无激光相机(config中禁用lidar的相机):</div> <div>0: 速度不可信</div> <div>2: 速度可信</div> <div>对于有激光相机:</div> <div>0: 速度不可信, 激光未修框</div> <div>1: 速度不可信, 但是修过框</div> <div>2: 速度可信, 而且修过框</div> <div>3: 速度可信, 但是没修框</div> <div>对于radar目标 (临时):</div> <div>0: 质量信息丢失</div> <div>1: 新目标</div> <div>2: 正常目标</div> |

|                                        |              |                                   |               |              |              |              |                                                                       |
|----------------------------------------|--------------|-----------------------------------|---------------|--------------|--------------|--------------|-----------------------------------------------------------------------|
|                                        |              |                                   |               |              |              |              | 3: 预测目标<br>4: 已删除目标<br>5: 新目标                                         |
| signal_light                           | -            | uint8                             | 1             | {0, 1, 2, 3} | 0            | -            | 目标信号灯:<br>0: 都不亮<br>1: 左转向灯亮<br>2: 右转向灯亮<br>3: 双闪                     |
| brake_light                            | -            | uint8                             | 1             | {0, 1}       | 0            | -            | 目标刹车灯<br>0: 没亮<br>1: 亮                                                |
| <del>cutin_state</del><br><del>s</del> | <del>=</del> | <del>array&lt;float32,4&gt;</del> | <del>16</del> | <del>=</del> | <del>=</del> | <del>=</del> | <del>压线概率、1s cutin概率、2s cutin概率、3s cutin概率</del>                      |
| door_state                             | -            | uint8                             | 1             | {0, 1, 2, 3} | 0            | -            | 目标开车门:<br>0: 没开<br>1: 开左边车门（相对于目标车头）<br>2: 开右边车门（相对于目标车头）<br>3: 开双向车门 |
| reid                                   | -            | vector<float32>                   | -             | -            | -            | -            | 目标 reid 神经网络特征                                                        |
| <del>polygon_base</del>                | <del>=</del> | <del>vector&lt;Vec2D&gt;</del>    | <del>=</del>  | <del>=</del> | <del>=</del> | <del>=</del> | <del>目标 polygon 自车坐标系下描述，逆时针凸包，开环（头尾不重复）</del>                        |
| project_boxes (vector)                 | sensor_id    | uint8                             | 1             | =            | $\theta$     | =            | 和激光目标关联的视觉传感器序列号                                                      |
|                                        | box          | BBox2D                            | 16            | =            | =            | pix          | 投影框                                                                   |
| pv_proj_point_base                     | -            | Vec3D                             | 12            | -            | -            | m            | pv投影参考点                                                               |
| extended_key                           | -            | vector<uint32>                    | -             | -            | -            | -            | 扩展字段键值<br><br>注：设置拓展字段注意已占据索引，避免冲突！                                   |

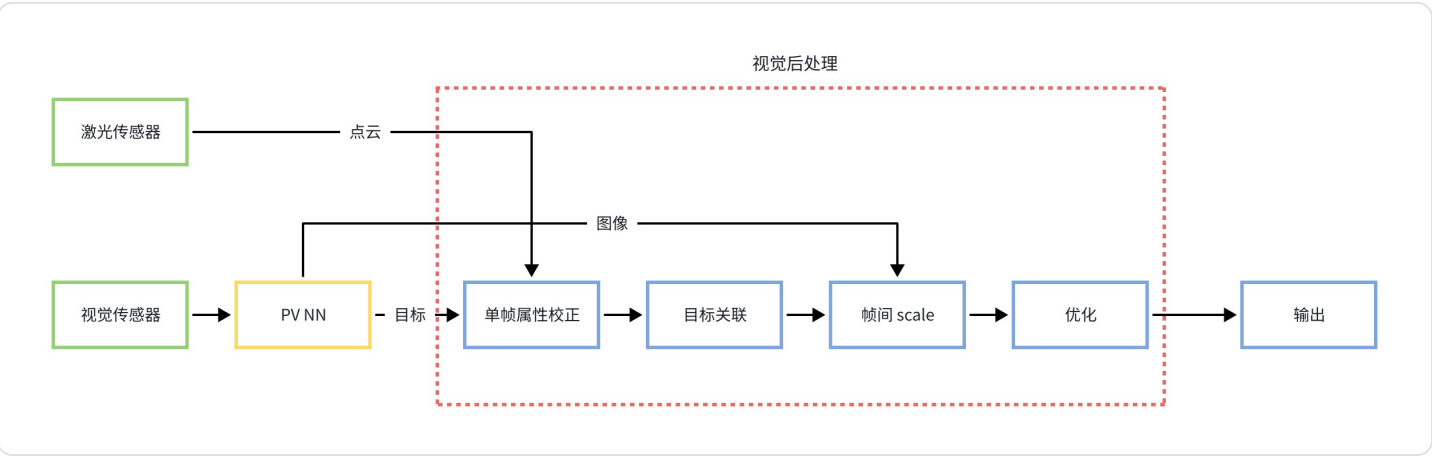
|                |   |                 |   |   |   |   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----------------|---|-----------------|---|---|---|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                |   |                 |   |   |   |   | <p>【bev_post出速度】 40: vx, 41: vy, 42: omega, 43: vx可信, 44: vy可信, 45: omega可信 [0不可信 1可信]</p> <p>【pv_nn截断系数】 50: proj_ratio[0] 51: proj_ratio[1]</p> <p>【pv_post静止置信度】 52: 静目标置信度 0 高 1 中 2 低</p> <p>【pv_post关联radar id】 53: 关联radar id</p> <p>【pv动静相关】 54: 动静标志置信度，越接近0越接近静止)</p> <p>【pv_post是否将2D框恢复至原始图像尺寸（适配新老版本）】 55: 不存在表示老版，存在表示新版</p> <p>【pv_nn开关车门挖掘】 56 左车门分数 57 右车门分数</p> <p>【pv_post 转向灯挖掘】 90: 低位刹车置信度 91: 低位刹车跳变置信度（不亮-&gt;亮） 92: 低位刹车跳变置信度（亮-&gt;不亮） 93: 高位刹车置信度 94: 左转置信度 95: 右转置信度</p> <p>【bev_nn出fov系数】 （60: fov_ratio)</p> <p>【Radar生存周期】 70: frame_count 71 : predict_count</p> <p>【Occupancy】 80: 疑似鬼影目标 0否 1是 81 横杆开闭（0 1 float） 82 横杆pitch角 83 （弃用，精度不足）横杆关联地图id84 occpancy关联 fusion 的 trackid（-1表示非动目标或无ID）</p> <p>85 横杆关联地图id高16位</p> <p>86 横贯关联地图id低16位</p> |
| extended_value | - | vector<float32> | - | - | - | - | 扩展字段键值对应的属性值                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

## 1.4 代码结构

代码块

```
1  mono3d/
2  |— algorithm/                # 算法工具
3  |   |— pv_nn/                # pv_nn核心代码
4  |   |— pv_post               # pv_post核心代码
5  |
6  |— component/                # 外部接口
7  |
8  |— config/                   # 配置文件
9  |   |— models/               # 模型文件, 分orin (aarch64) ,PC(x86)
10 |
11 |— dag/                      # 启动dag
12 |
13 |— luanch/                   # 启动luanch
14
```

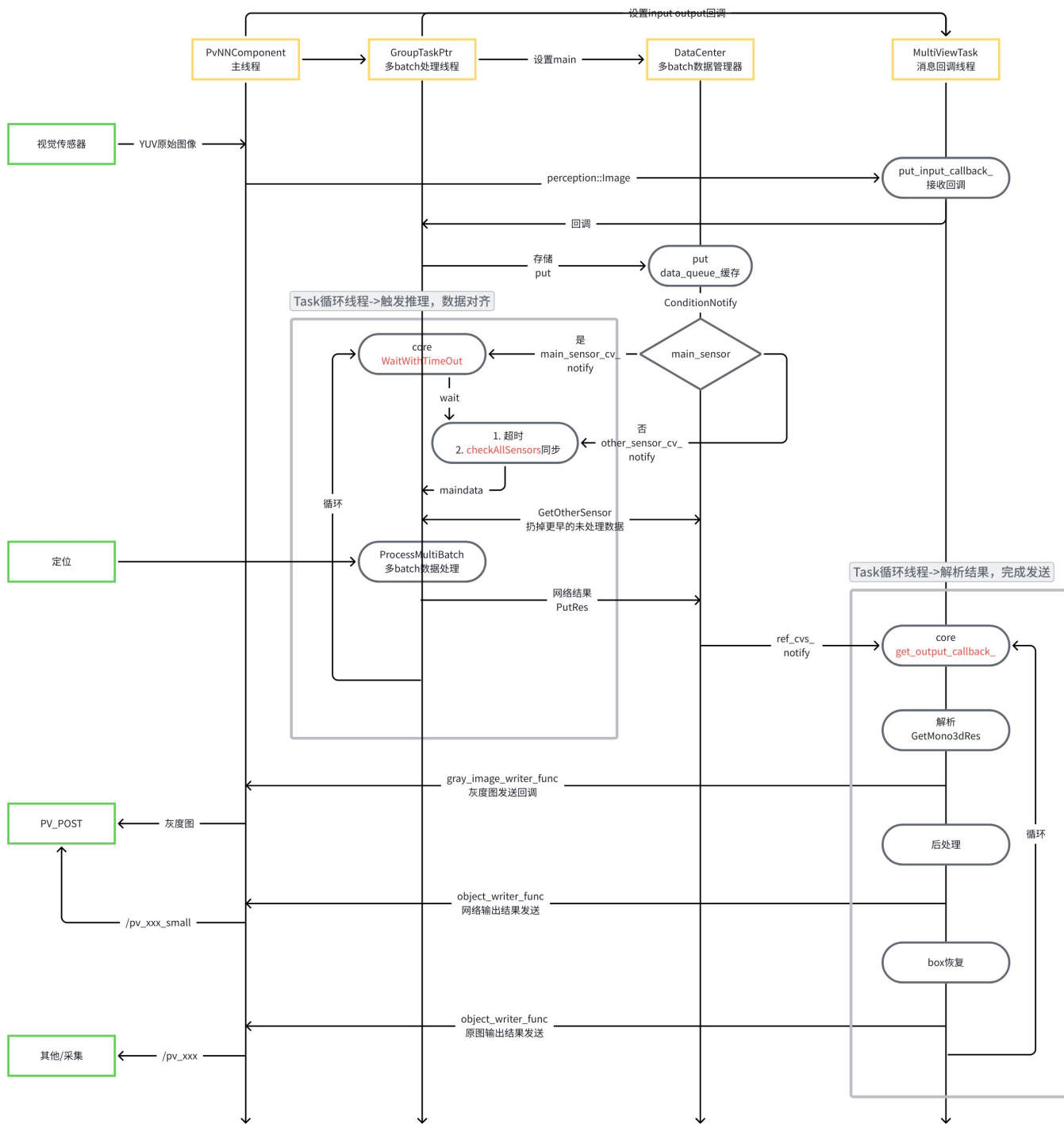
## 1.5 总体框图



## 2. PV\_NN

### 2.1 完整数据流

- pv\_nn的核心是一个模板DataCenter机制，使用该机制来实现多batch的超时等待
- 单batch时不再存在WaitWithTimeOut的线程通信机制，改为一个简单的get-wait机制，其他基本逻辑一致



## 2.2 对外接口PvNNComponent

- 主要用于task生成、消息订阅和输出回调

### 2.2.1 task生成

- 读取参数服务器的use\_rear\_camera\_data参数判断是否允许使用后视针孔相机
- 初始化图像降频控制器 (pv\_nn的时序模型仅支持固定10HZ)
- 根据config中group\_idx, is\_main\_sensor参数控制group\_ptr生成, 每个group都只唯一的group\_ptr, 用于推理

- 每个sensor有位移的pinhole\_task\_ptr，这个是跟随group一次性初始化的，并不会单独初始化，在接口中只用于设置输入回调和输出回调

## 2.2.2 消息订阅

- 定位消息存储在EgoPoseInfo单例中
- 图像订阅有两种方式，一种是零拷贝的sensor\_msgs::ImageLowDelay，一种是sensor\_msgs::Image，区别是转换得到的perception::Image是否使用了深拷贝，转换后下游无感
- 图像输出针孔有原图结果、灰度图像、网络结果三个输出，其中两个用于后处理，鱼眼仅有原图结果一个输出

## 2.3 推理部分GroupTaskPtr

### 2.3.1 初始化Init

#### 2.3.1.1 Task初始化->关键是标定初始化

- mono3d网络推理并不是使用标准的去畸变图，为了训练推理的一致性、稳定性和泛化性，相机都使用了固定旋转外参以及不同的展开模型虚拟内参

### 1. 针孔



- 针孔使用的是**等距柱面展开**，即将一个单位柱面上 $R=1$ 的点按等角投影投影到图像上，这样每条每条射线都成像为一个点，描述为  $(\theta, h)$

令图像上两个点为 $(u, v)$

则真实角度 $\theta = (u - f_x)/c_x, h = R \tan \phi = \tan \phi = (v - f_y)/f_y$

由于角度等距，则对应柱面相机坐标系中射线方向向量为，该射线与真实世界射线平行

$$X = f_x * \sin(\theta)$$

$$Y = f_y * \tan \phi$$

$$Z = f_x * \cos(\theta)$$

- 虚拟旋转中认为**roll**为-90°，**pitch**为0°，即认为光轴水平（并且相机坐标系y向下）
- 虚拟旋转的**yaw**则是将每个相机转到了固定安装位置上，

```
const std::map<uint8_t, float> rect_yaws = {  
    ... {static_cast<uint8_t>(perception::SensorID::CAM_FRONT_WIDE), -1.57080},  
    ... {static_cast<uint8_t>(perception::SensorID::CAM_FRONT_NARROW), -1.57080},  
    ... {static_cast<uint8_t>(perception::SensorID::CAM_LEFT_BACK), 0.81331},  
    ... {static_cast<uint8_t>(perception::SensorID::CAM_LEFT_FRONT), -0.61086},  
    ... {static_cast<uint8_t>(perception::SensorID::CAM_RIGHT_BACK), 2.32828},  
    ... {static_cast<uint8_t>(perception::SensorID::CAM_RIGHT_FRONT), -2.53072},  
    ... {static_cast<uint8_t>(perception::SensorID::CAM_BACK), 1.57080},  
};
```

- 整体流程是：创建和网络输入等大的remap图像，生成虚拟内参，根据原始图像去畸变后的图像边缘真实射线角度确定纵横fov，生成能覆盖完全fov的虚拟内参，再从remap图像上每一个像素点，图像坐标->方向向量->虚拟外参旋转->归一化相机坐标->加畸变->图像坐标，得到原始图像上的一个插值点，生成remap图像

## 2. 鱼眼





- 鱼眼使用的是**球面展开图**，即将一个用经纬度描述的单位球面，并将球面角度展开后等间距投影到平面上

归一化球面（深度为1平面）坐标为： $\theta = x = \frac{u - c_x}{f_x}, \varphi = y = \frac{v - c_y}{f_y}$

相机坐标系下的方向向量为： $X = \sin \theta \cos \phi, Y = \sin \theta \sin \phi, Z = \cos \theta$

- 虚拟旋转中认为**roll为-90°**，**pitch为0°**，即认为光轴水平（并且相机坐标系y向下）

```
std::map<uint8_t, float> rect_yaws = {
    {static_cast<uint8_t>(perception::SensorID::CAM_FISHEYE_FRONT), -M_PI / 2},
    {static_cast<uint8_t>(perception::SensorID::CAM_FISHEYE_LEFT), 0.0f},
    {static_cast<uint8_t>(perception::SensorID::CAM_FISHEYE_BACK), M_PI / 2},
    {static_cast<uint8_t>(perception::SensorID::CAM_FISHEYE_RIGHT), M_PI}};
```

- 整体流程是：创建和网络输入等大的remap图像，生成虚拟内参，根据原始图像去畸变后的真实射线角度确定横纵fov，生成能覆盖完全fov的虚拟内参，再从remap图像上每一个像素点，图像坐标->方向向量->归一化相机坐标->加畸变->图像坐标，得到原始图像上的一个插值点，shegnchengremap图像

### 2.3.1.2 engine初始化

- 区分aarch64和x86平台进行模型加载，同时aarch64平台上使用统一内存映射将cpu内存和gpu内存映射use\_Unified\_Address
- 针孔有**batch2** batch1两种模型，鱼眼有batch1 **batch4**两种模型



### 2.3.1.3 cuda内存初始化

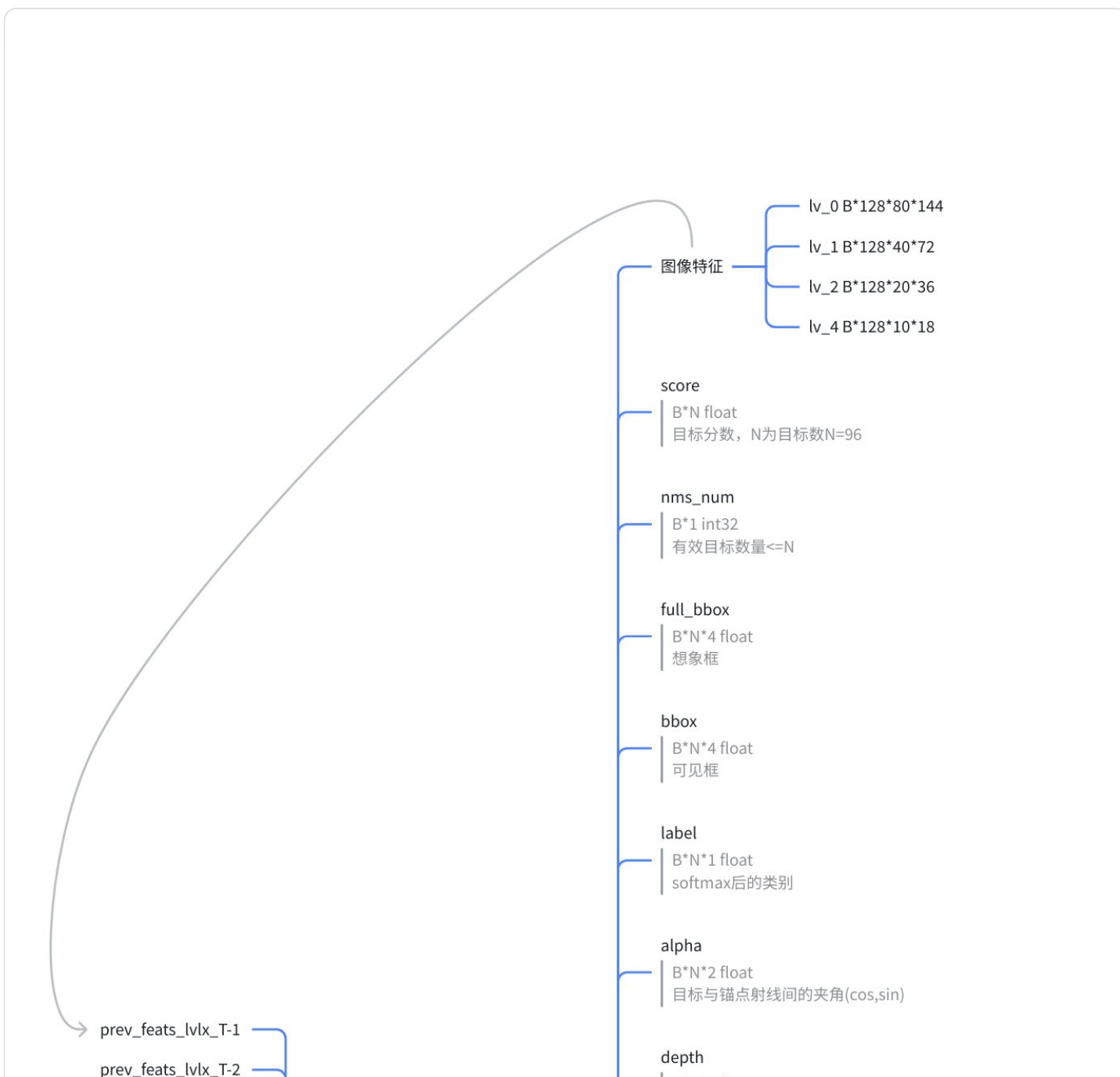
- pv\_nn中使用一个自定义的cuda算子**cuda\_nv12\_remap\_norm.cu**进行图像解码、remap等操作,所以需要提前开辟空间用于算子操作

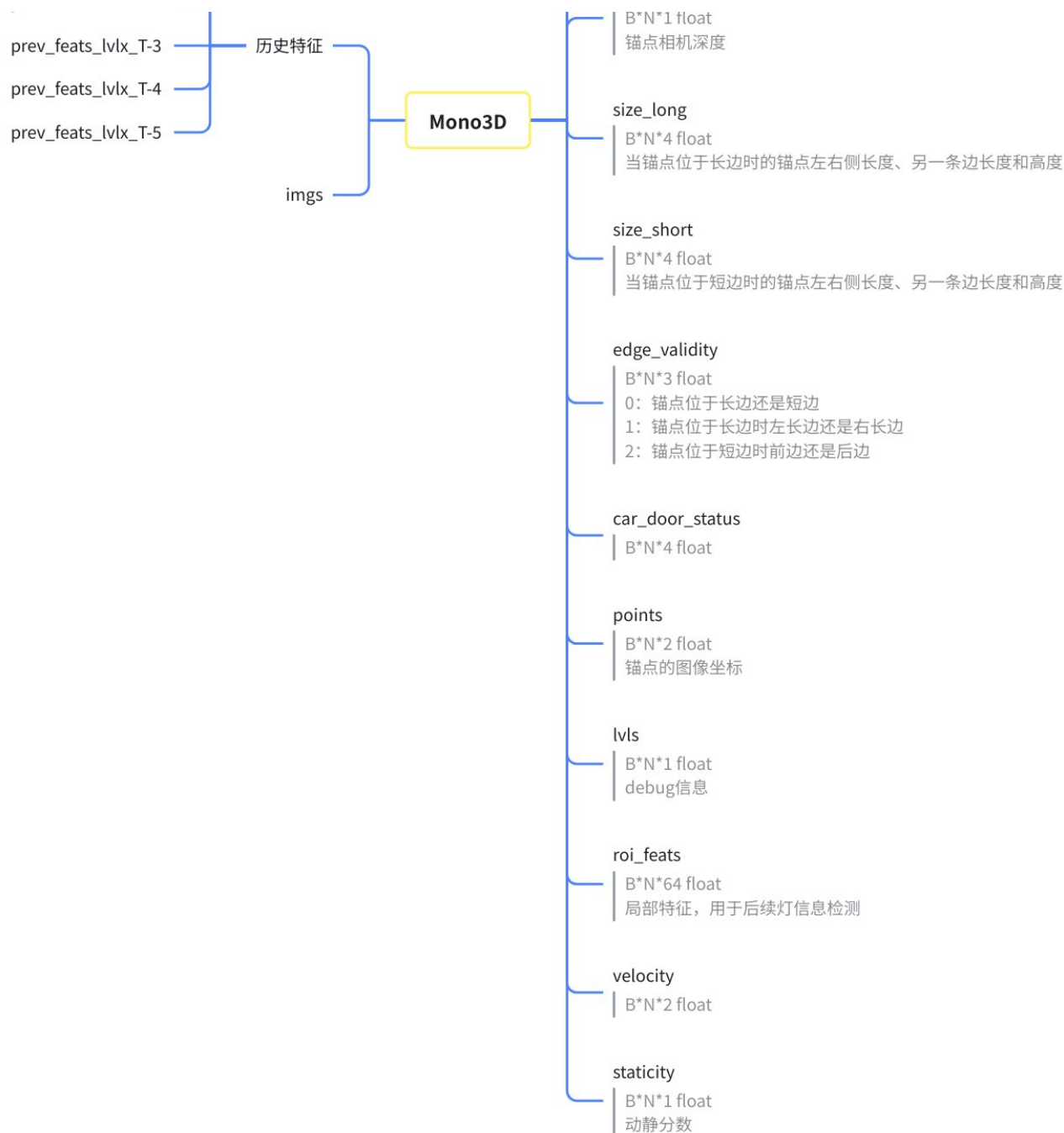
cuda\_img\_yuv\_ 编码图像 cuda\_map\_xy\_remap的映射表 cuda\_gray\_gpu上的gray内存  
cuda\_image\_nchw\_ 网络输入格式 cuda\_workspace\_工作临时变量区 此外还有用于cpu gpu跨设备传输的cuda\_img\_yuv\_host\_和cuda\_gray\_host\_用于控制异步拷贝

## 2.3.2 预处理

### 2.3.2.1 网络结构

- 网络输入包括5帧历史特征累帧和当前帧的NCHW图像，每个feature单元包括4层不同尺度feature，上一帧的feature输出会作为当前帧的feature\_0输入，同时上一帧的feature输入顺延输入，**feat间间隔为固定的100ms**





### 2.3.2.2 图像remap PreProcessImage

- 将yuv422/nv12编码的cpu图像异步拷贝到cuda\_img\_yuv
- PreprocessImageNv12 中构建参数进入YUVRemapAndSplit算子，经过remap、归一化和通道重组后cuda\_image\_nchw\_用于imgs输入
- 将gray图像异步拷贝到cuda\_gray\_host\_用于后续输出

### 2.3.2.3 时序特征拼接BuildPrevFeats

- 利用了cuda的setTensorAddress机制，不是向固定的gpu内存中拷贝输入，而是直接切换网络处理的内存地址。可以认为网络初始开辟了六块等大的GPU内存，每块内存对应了一块feature，其中一块用于输出，五块用于传递输入。

1. **不丢帧时**: 对于当前帧, 上一帧的输出`feat_out`, 其实就是当前帧的`feat_1`, 上一帧的`feat_x`则是下一帧的`feat_x+1`, 最后一帧`feat_5`则置零后用于输出, 对应下图第一幅图
2. **丢N帧时**: 此时上一帧和当前帧间隔为 $(N+1)*100\text{ms}$ , 则`feat_out`应该对应了当前帧输入的`feat_{(N+1)}`, 同时上一帧`feat_x`就变成了当前帧的`feat_{(x+N+1)}`, 根据轮巡内存, 保证`feat_5`仍然作为输出, 所以空出来的上一帧的`feat_{(5-t)}`,  $t=1\sim N$ 就作为循环内存顺延到当前帧调过的`feat_t`中 (实际上`miss_frame>1`时在后续解码中速度将被无效掉)



- 当前帧进行了处理, 就将当前帧时间戳记录下来
- 如果存储图像的话就把`image_nchw`拷贝回cpu, 然后重新恢复成rgb图像

### 2.3.3 后处理

- 从`cuda_gray_host_`获取灰度图->只有针孔用于`pv_post`
- 根据batch信息、尺度信息和类型信息将每个输出按名称拷贝到CPU的指定内存中

- 将结果传入data\_center，触发multitask的接收回调

## 2.4 解码输出部分 MultiTaskPtr

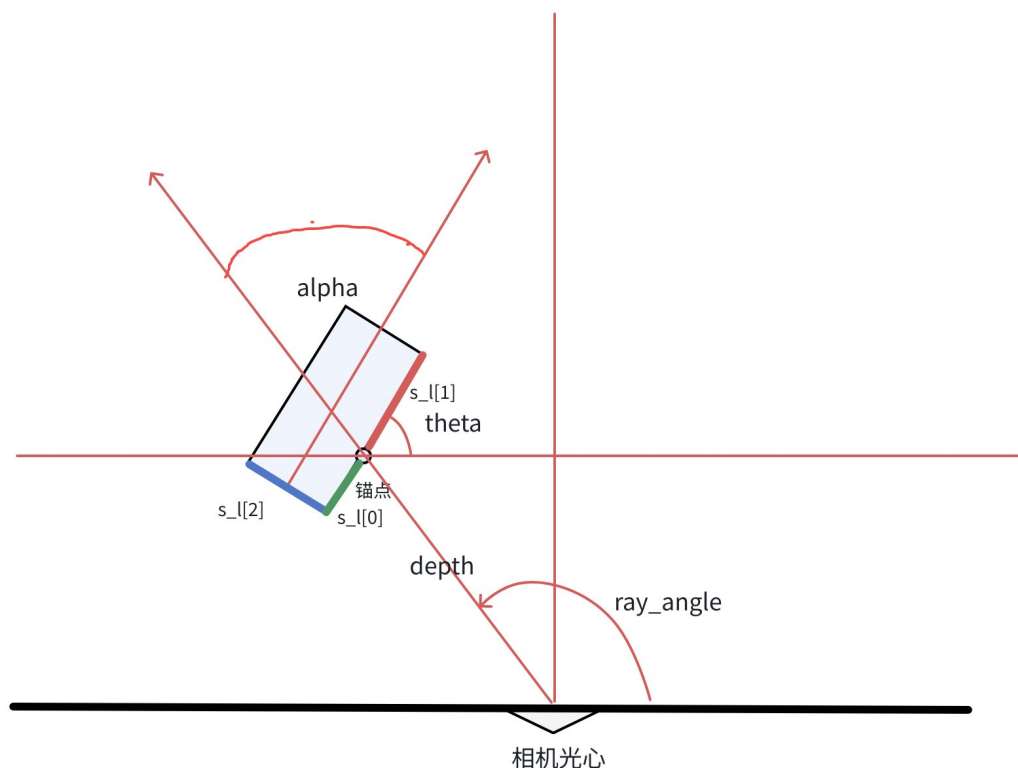
- FrameMiss根据输出信息获取当前sensor历史5帧（500ms）丢失多少帧，超过一帧将没有速度
- GetMono3dRes将字节流转换为结构化的Mono3dObject
- 提前发送灰度图，pv\_post灰度图接收解析存在一定delay

### 2.4.1 结构化目标



**Mono3D使用了锚点回归，目标3D信息回归的基本都是与锚点的相对关系，并不直接回归直接的3D信息**

下图绘制了当前使用的柱面和鱼眼模型中网络的输出示意



#### 2.4.1.1 获取锚点视线角和锚点相机3D坐标 GetProjPt

- 根据锚点的像素坐标point，计算视线角和俯仰角，并且右侧为0°左侧为180°

$$ray = \pi/2 - (p_x - c_x)/f_x, phi = (p_y - c_y)/f_y$$

- depth有放缩，并且直接学的是一个解耦焦距的量（大约可以理解为多少倍焦距），所以需要恢复成物理单位，锚点3D坐标为：

$$X = depth * \sin(\theta) = depth * \cos(ray)$$

$$Y = depth * \tan\phi$$

$$Z = depth * \cos(\theta) = depth * \sin(ray)$$

- 同时根据想象框中点计算中心点phi角bbox\_center\_phi\_angle，直接计算得到中心点的z值

### 2.4.1.2 朝向、尺寸和中心点恢复 DecodeMono3d

这里关键是edge\_validity, alpha,, s\_l和s\_s, edge\_validity表示锚点位于长边短边、左长边右长边、前短边后短边, alpha表示目标朝向和锚点视锥之间的夹角, s\_l是锚点位于长边时锚点左侧长度, 和锚点右侧长度, 短边长度以及高度, 朝向, s\_s同理, 尺寸中心点依赖于锚点和长短边的关系

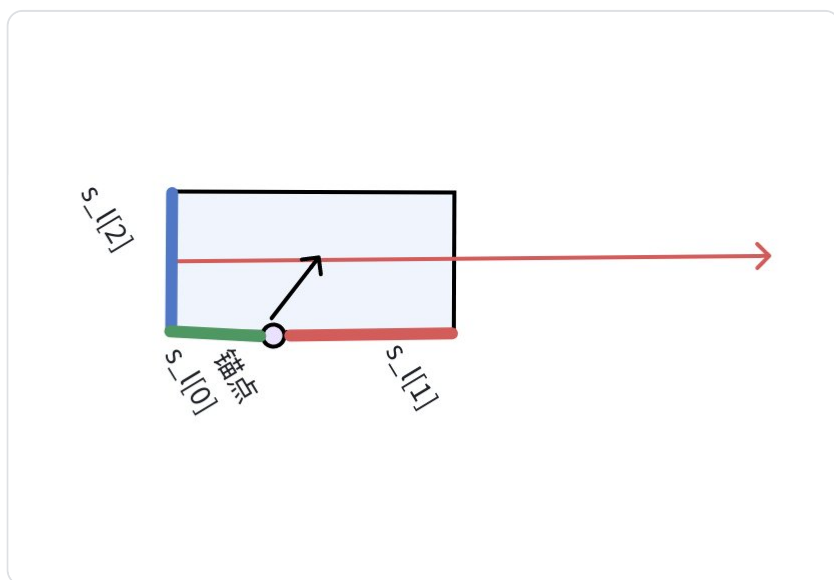
- 小目标没有明确长短边, 所以行人**高度**使用长短边锚点估计的平均值, **长宽**是定值, **中心点位置**直接根据射线方向延长0.5l 0.5w, 三角牌高反牌**尺寸**均使用定值,**中心点位置**直接根据射线方向延长0.5l 0.5w, 行人网络是直接回归了真实夹角, **朝向**为

$$theta = ray + alpha$$

- 小动物尺寸变化较大, 没有专门监督长短边, 2D精度较高, 所以使用2D框宽度反投影**宽度** (长度=宽度), **高度**利用2D高反投影得到高度,**中心点位置**直接根据射线方向延长0.5l 0.5w, 没有朝向
- 对于长目标, 其alpha训练存在一个归一化, 实际值域是 $[0, 2\pi]$ , 但是因为atan无法表征, 所以在转换后失去了其实际方向信息, 需要根据网络训练的逻辑进行一次真实朝向恢复,

#### 1. 位于长边, 长为s\_l[0],s\_l[1]分段组合, 短边为直出s\_l[2],高度为s\_l[3]

在以车体中心为原点, 向右为x向前为z的坐标轴上, 如果锚点位于右边, 那么中心点相对锚点的X偏移量 $X=(s_l[1]-s_l[0])/2$ , Z的偏移量 $Z=0.5*w$ , 网络假定此时真实alpha角应该在 $0-M\_PI$ 之间, sin为正数, 所以要将其恢复到sin为正, 再进行  $theta = ray + alpha$ , 利用朝向将位于目标坐标系下的偏移量旋转到相机坐标系下



如果锚点位于左边, 那么中心点相对锚点的X偏移量 $X=(s_l[0]-s_l[1])/2$ , Z的偏移量 $Z=-0.5*w$ , 网络假定此时真实alpha角应该在 $-M\_PI-0$ 之间, sin为负数, 所以将其恢复到sin为负, 再进行  $theta = ray + alpha$

2. 位于短边，长边为 $s\_short[2]$ ，短为 $s\_short[0]$ ， $s\_short[1]$ 分段组合，高度为 $s[3]$

同上，可以得到X Y在无旋转的偏移量

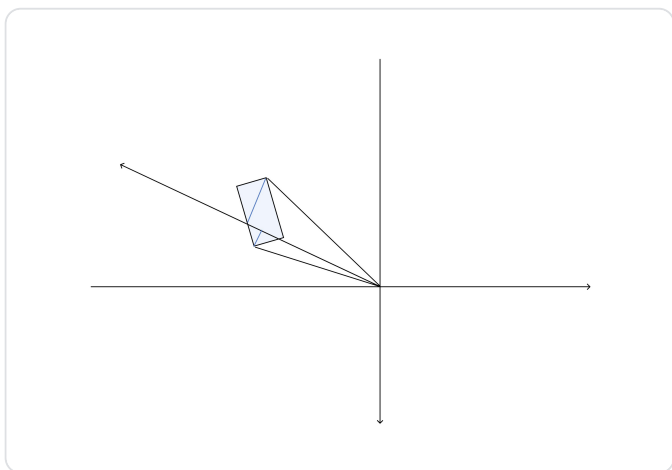
得到偏移量后再乘上旋转恢复中心点，再转到车体坐标系作为输出

$x\_cam = \cos(\theta) * X - \sin(\theta) * Z + proj\_pt\_x$

$y\_cam = \sin(\theta) * X + \cos(\theta) * Z + proj\_pt\_y$

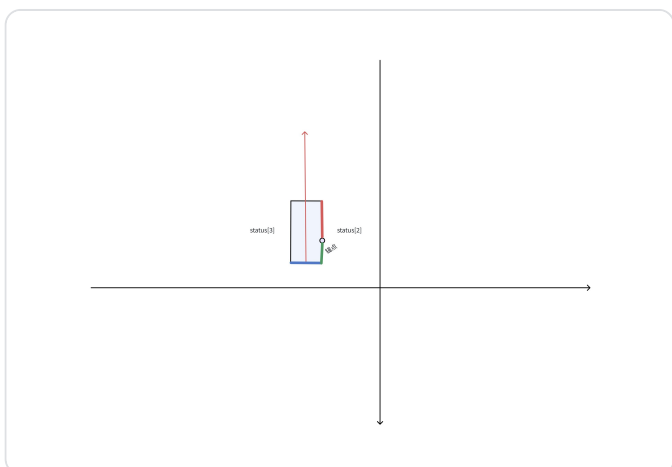
### 2.4.1.3 计算fov外投影比例 CalcProjRatio

- 根据fov计算左右边界点的X Y 射线，利用叉乘判断四个角点的fov内外关系以及计算四个角点在左侧视场边界内外的长度比和右侧视场边界内外长度比



### 2.4.1.4 开关车门状态恢复 PostProcCardoor

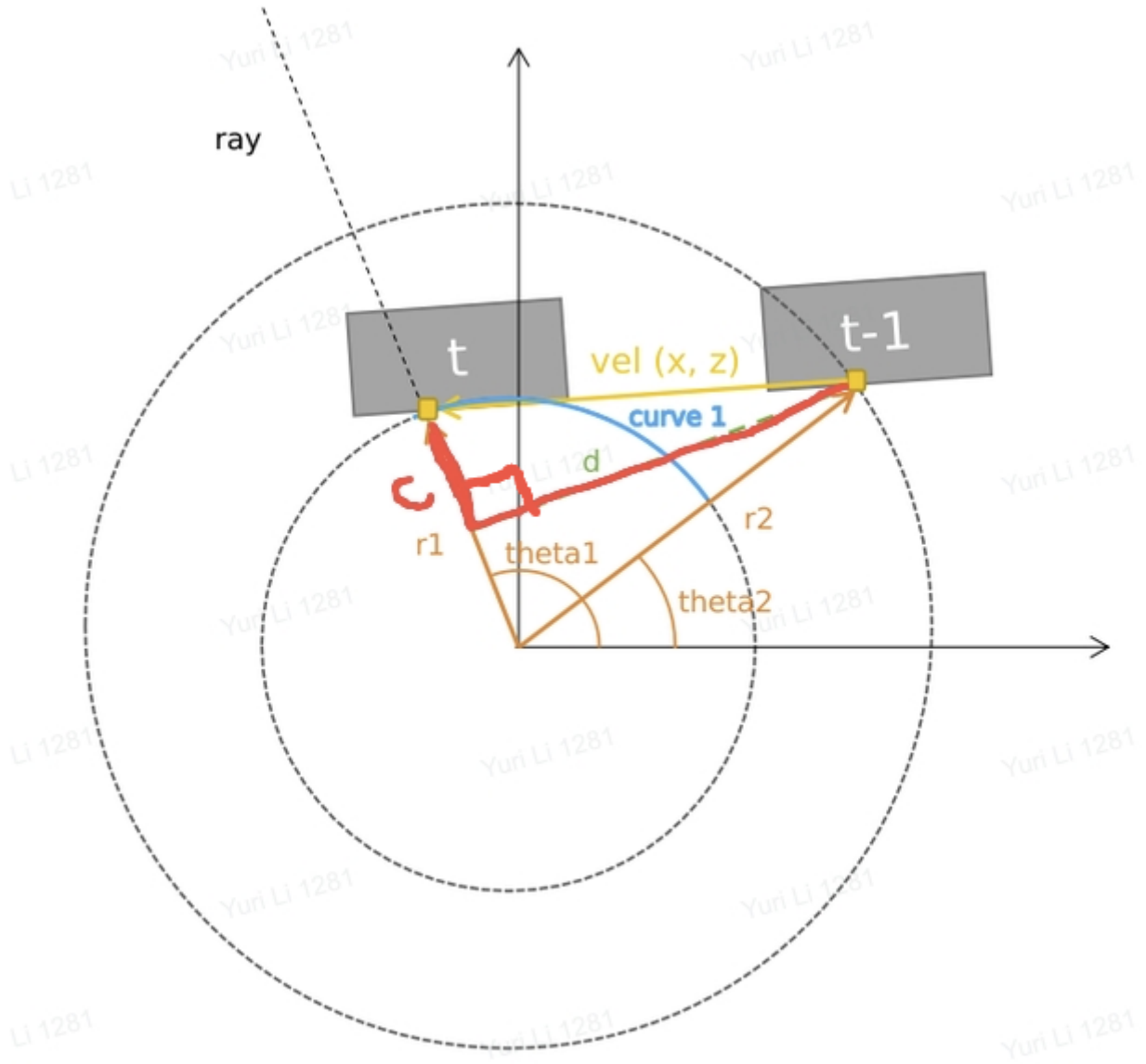
- 其实就是要判断哪个是目标的左边车门哪个是目标的右边车门
- mono3d开关车门共输出了4个值，0 1是锚点在短边时，以车体方向的左边和右边的开关车门状态，2和3是当锚点位于长边时，可见边和另一边的。锚点能标识可见边的位置，所以当锚点位于目标朝向左长边时，可见边就是左侧，所以 $left=2, right=3$ ；当锚点位于朝向右侧时，可见边是右侧边，所以 $left=3, right=2$ ，需要调整。当锚点位于短边时，0 和1就分别是相机视角的左边和相机视角的右边，所以当锚点位于车头时，说明车头朝向相机，那么相对于目标的左右就要反向



- 用0.8作为阈值，输出基于目标朝向的左右开关车门信息

### 2.4.1.5 速度计算 CalcVelocity

- 只有丢帧小于一帧才会算这个速度
- 网络输出的速度是两个几何量  $vel_{reg} = [x, z]$  目，前针孔柱面模型和鱼眼展开模型都改为了方案3，也就是回归的是目标锚点沿着当前帧径向和切向的相对运动速度



1. 计算锚点的径向向量和切向向量：

$$\hat{r} = [pt_{cam_x}^t / d_1, 0, pt_{cam_z}^t / d_1], \text{ where } d_1 = |pt_{cam_{x,z}}^t| = \sqrt{(pt_{cam_x}^t)^2 + (pt_{cam_z}^t)^2} + \epsilon$$

$$\hat{t} = [-r_1, 0, r_0]$$

2. 计算得到相机坐标系下的相对速度：

$$vel_{cam} = (x * \hat{t} + z * (f/500) * \hat{r}) * 10$$

$$vel_{cam\_base} = R_{cam2veh}^t vel_{cam}$$



后续还利用odom坐标系算了一个速度，使用上述vel\_cam直接时间递推得到，锚点在odom下的两点位置，算了一下差分，可以认为是目标在当前自车坐标系下的绝对速度，将cam下的转换的base速度和odom下得到的base速度分别填写在了vel\_cam3d和vel\_odom3d，输出时一个填在了acceleration\_base一个填在了velocity\_base

### 2.4.1.6 动静计算GetStaticity

- 直接评分输出

### 2.4.2 目标滤除PostFilterObjs

- 根据fov外投影比例、full\_box框遮挡比例和分数打标，nn\_state==1表示低分,==2表示截断，==3表示遮挡
- 再做一个全类别NMS

### 2.4.3 目标尺寸恢复ResizeObjs

- 鱼眼相机并没有恢复到原图上，只是通过resize和offset恢复到了原图尺寸上，因为并没有后续节点使用
- 针孔相机要恢复到带畸变原图上用于lidar-post节点点云投影使用，使用的不是四个角点，因为同形畸变中角点并不是外包络点，而是**四边中心点**，对中心点加畸变在resize回原图

### 2.4.4 输出

- 除了常规的网络属性输出，额外增加了五个拓展输出

mono3d\_obj.size\_proj\_ratio 截断投影比例，用于pv\_post额外判断截断

mono3d\_obj.staticity 动静分数，

mono3d\_obj.car\_door\_status\_postproc 开关门评分，用于后续融合使用

mono3d\_obj.roi\_feature ROI feture，用于pv\_post中关联后进行灯信息推理

## 2.5 存图

- 将group\_task\_ptr和multi\_task\_ptr中的// #define SAVE\_IMAGE解出注释会存储推理图像及其推理结果



## 3. PV\_POST

### 3.1 数据流

- PV\_POST是个简单的单触发多单例模式，每个sensor间相互独立，由单一的pv\_nn消息进行触发，定位、点云、radar、图像等则由公共单例缓存，使用时处理线程查询获取指针。



### 3.2 外部接口及缓存管理

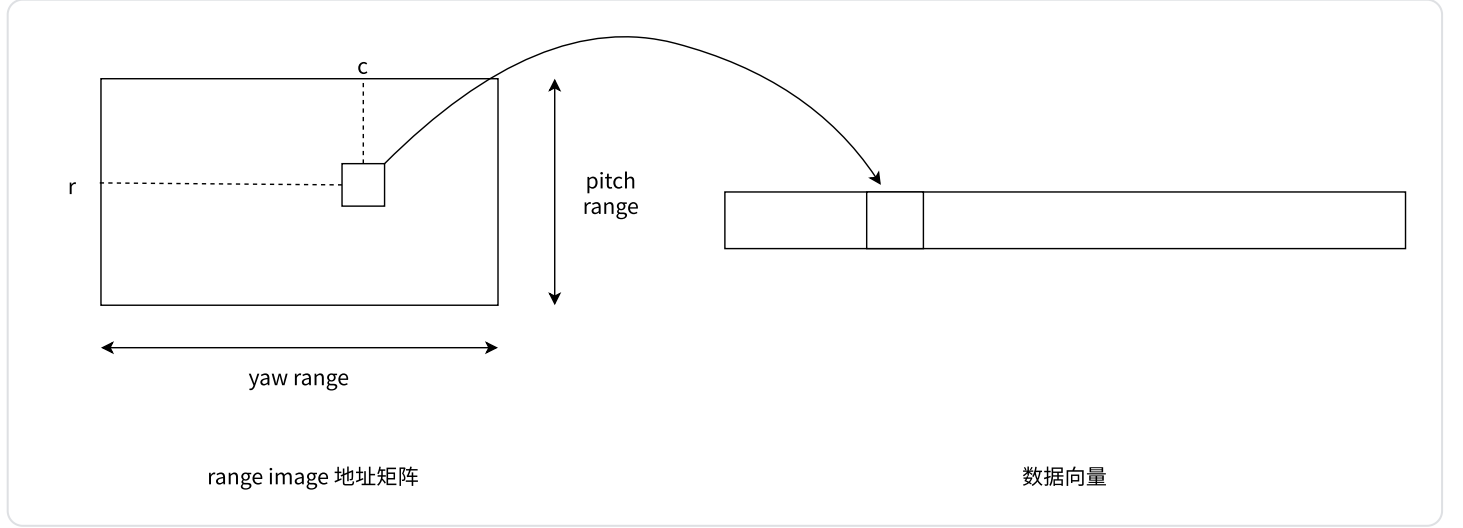
#### 3.2.1 lidar缓存管理 RangImageCacheCenter

##### 1. 并发管理

- 该缓存用于管理lidar点云数据及**RangImage**数据，RangImage可以在该单例中自己计算
- 由于rangeimage和pointcloud都需要大块连续内存进行存储和访问，如果后期开辟会存在严重的耗时问题，所以使用提前开辟**对象池**内存进行循环缓存，并且存储时也要直接原地存储，不要进行拷贝。读取则不进行长期锁，依赖长缓存路径和外部处理速度避免处理时存储踩踏。
- 并发机制由**全局锁**和**对象局部锁**构成。存储时先锁掉全局锁取出局部对象，给局部对象加锁后释放全局锁，外部处理结束后需要显式释放局部对象解锁；读取时则锁掉全局锁进行查询，如果要进行点云和range额外对齐则需要锁掉局部锁，拿出真实对象后解锁。

第一个矩阵：range image 地址矩阵，该矩阵的每个元素填入对应数据向量的首地址

根据激光的 FOV，设定 range image 的范围， $\text{yaw range} \in [60^\circ, -60^\circ]$ ， $\text{pitch range} \in [12.5^\circ, -12.5^\circ]$ ，如下图所示



yaw 方向上采用分辨率为  $\text{res}_{\text{yaw}} = 0.2^\circ$ ，共  $\lceil \frac{\text{yaw}_{\text{max}} - \text{yaw}_{\text{min}}}{\text{res}_{\text{yaw}}} \rceil$  列

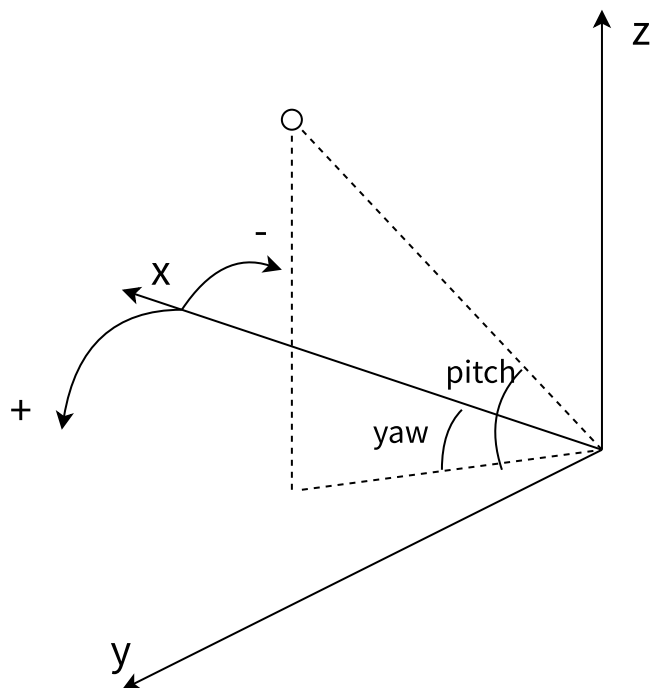
pitch 方向上采用分辨率为  $\text{res}_{\text{pitch}} = 0.2^\circ$ ，共  $\lceil \frac{\text{pitch}_{\text{max}} - \text{pitch}_{\text{min}}}{\text{res}_{\text{pitch}}} \rceil$  行

假设某点云在自车坐标系下经过运动补偿后的坐标为  $\mathbf{p}_v = \begin{bmatrix} x_v \\ y_v \\ z_v \end{bmatrix}$ ，先计算其在激光坐标系下的

$\mathbf{p}_l = \begin{bmatrix} x_l \\ y_l \\ z_l \end{bmatrix} = \mathbf{R}_{v2l} \mathbf{p}_v + \mathbf{t}_{v2l}$ ，然后计算  $\text{yaw}, \text{pitch}$

$$\text{yaw} = \arctan \frac{y_l}{x_l} = \arcsin \frac{y_l}{\sqrt{x_l^2 + y_l^2}}$$

$$\text{pitch} = \arcsin \frac{z_l}{\sqrt{x_l^2 + y_l^2 + z_l^2}}$$



然后计算该数需要落入的坐标

$$r = \left\lfloor \frac{pitch_{max} - pitch}{res_{pitch}} \right\rfloor$$

$$c = \left\lfloor \frac{yaw_{max} - yaw}{res_{yaw}} \right\rfloor$$

将该点云的 index 填入（range image 所对应的首地址 + 当前该坐标内已经存储的点云数量）所对应的数据向量内

### 3.2.2 图像缓存 DataWareHouse

- 使用了**内存池存储**，内部实现了类似于share\_ptr的ref\_count机制和先入先出缓存清理，通过设立足够长缓存序列来避免读写冲突

### 3.2.3 初始化

- 根据参数服务器的based\_lidar\_name参数确定当前使用哪个真实的config，主要影响激光点云接收，并**读取标定**
- 完成对**公共信息**，激光、定位、radar消息的订阅，同时对涉及到的缓存管理器进行初始化
- 每个sensor对应一个**PvPostInterfacePtr**，PvPostInterfacePtr是处理主线程，管理pv消息的订阅发送以及对实际处理函数的触发
- **MultiViewPostProcessTask**是实际的处理类，因为pv\_nn下发的结果是remap后的结果，所以处理实际使用的标定要按照与pv\_nn相同方式进行转换（柱面）ApplyScaleIntoK
- 灰度图只用于scale计算速度，所以部分场景并不需要接收图像，节省内存只有需要scale的enable\_scale才会创建内存存储图像

- 部分sensor需要出转向灯刹车灯，创建灯信息推理引擎

## 3.3 实际处理 MultiViewPostProcessTask

### 3.3.1 消息转换ConvertToMono3dObjs

- 将ObjectInner转换成后续使用的Mono3dObject格式表示单帧观测
1. 根据分数和位置信息滤除对下游无效的目标
  2. 根据行人2D框和背景车辆2D框的包含关系以及底边高度关系滤除**车上站立行人**
  3. 根据锥桶的几何属性、2D和背景车辆包含关系以及估计3D情况滤除**运动锥桶**
  4. 解析转换为Mono3dObj

通常内容直接拷贝，点云速度在上游给出有效（is\_box\_refine）时填写，否则置空

拷贝64位roi feature

使用更严格的策略，结合上游的nn\_state和2D box情况给截断、遮挡目标**打标**，低分标记为valid=false

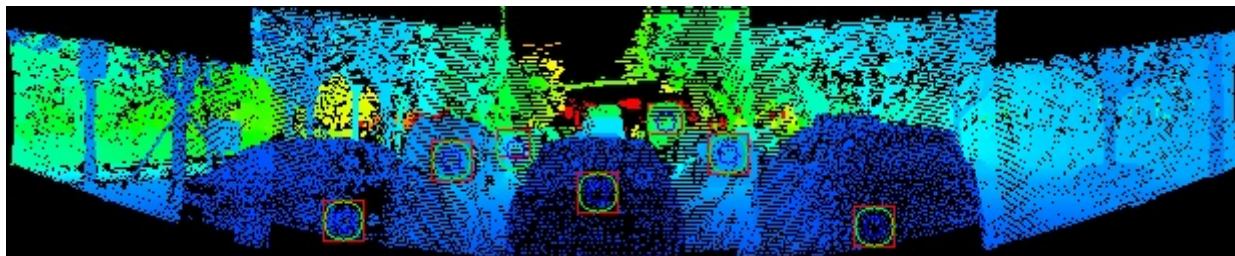
将上游传入的锚点3D坐标恢复2D点，并记录当前帧锚点到可见框的偏移量，用于后续位置调整

- 转换为内部格式后**排序**，优先处理动目标，优先处理近距离目标，保证后续点云关联的准确性
- 从锚点3D坐标中恢复目标在相机坐标系下的深度，用于后续scale深度优化

### 3.3.2 点云目标修正RefineByLidarDistanceWithRangeImage

#### 1. 基础关联

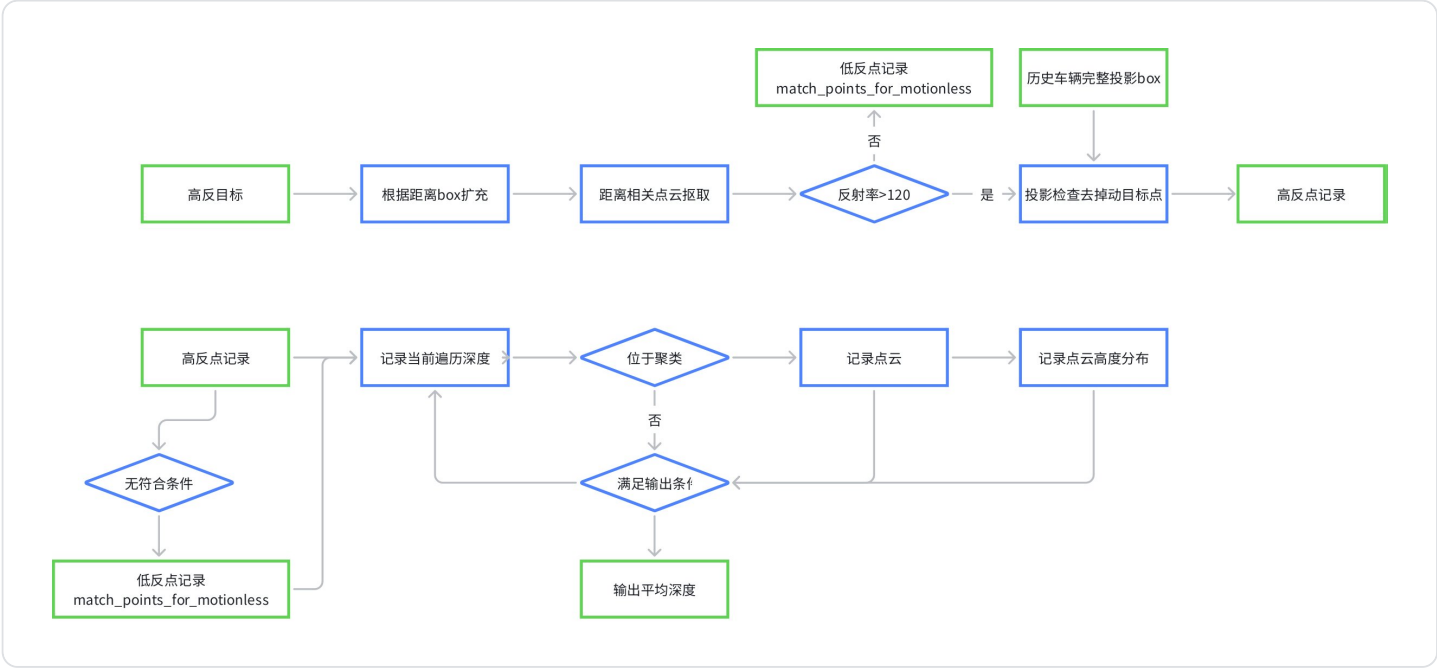
- 将目标的2D框投影至rangeimage上，根据一定规则抠取符合条件的点云，求取平均深度校正目标观测的距
- 将锚点及2D框的左右角点利用网络深度投影到base坐标系下，再投影到range图像的栅格中栅格坐标，使用锚点为中心绘制一个10\*10的window和左右角点构成的box取交集，就是实际抠取的rangeimage区域



- 根据先验mono3d的误差通常小于15%，根据这个逻辑抠取点云，并将点云按照距离排序，抠取完成后进行一次距离聚类，要求聚类中的点大于点数阈值认为关联有效，以加权平均距离作为校正距离

#### 2. 锥桶关联

- 锥桶的点云关联更加复杂，下图为点云关联流程图。当距离较近时，通常激光雷达有较多高反点，但在远距离，由于锥桶位置误差较大以及雷达性能限制，通常只有少量高反点、低反点甚至没有点云，还很容易和周围其他目标、地面标志等误关联。高反关联时锥桶会记录is\_update\_depth\_，低反关联则记录为is\_update\_depth\_low\_confidence\_



- 深度校正时用新的深度校正锚点射线位置，然后再根据中心点差值进行偏移

### 3. 锥桶存在性处理

- 一个检出的静目标只要所在位置有点云存在，就不应该消散，所以额外增加了一次处理对历史检测到的中高置信度锥桶进行一次简单的点云查找，但是不进行修正，只打标，弥补一些漏检误消散问题



#### 3.3.3 图像处理 AddImageInfo

- 对于需要使用scale优化的目标，需要提取图像区域img\_template\_ptr
- 为了避免关键点计算混入大量遮挡目标，根据观测目标的下底边大小关系（y越大越靠近自车），使用更近目标的box给被遮挡目标的图像区域打标cover\_mask\_ptr，同时使用这种方式再更详细记录目标的遮挡比例cover\_ratio





### 3.3.4 目标关联 TrackingObjs

- 目标关联基础使用**预测IOU和贪心匹配**，但由于预测过程中自车颠簸定位插值不准时很容易导致远距离目标Y方向2D框发生较大偏差，而pitch角带来的偏差是政府图像y方向的偏移，所以可以少量可以匹配目标的Y方向差值进行全局补偿，所以进行一次粗匹配计算Y方向补偿量



#### 3.3.4.1 匹配分数计算 MatchScore

- 大多数情况都不考虑距离关系，只使用预测IOU相当于视锥关联，但部分情况直接不允许关联：
  - pv网络类型观测不可能映射不关联
  - 车辆类目标朝向误差较大增加距离校验
  - 低分、遮挡目标关联时增加距离校验
  - 类别不同的车辆类目标增加距离校验
  - 速度较大时速度方向大于90°不关联

6. 锥桶类目标距离过远不关联

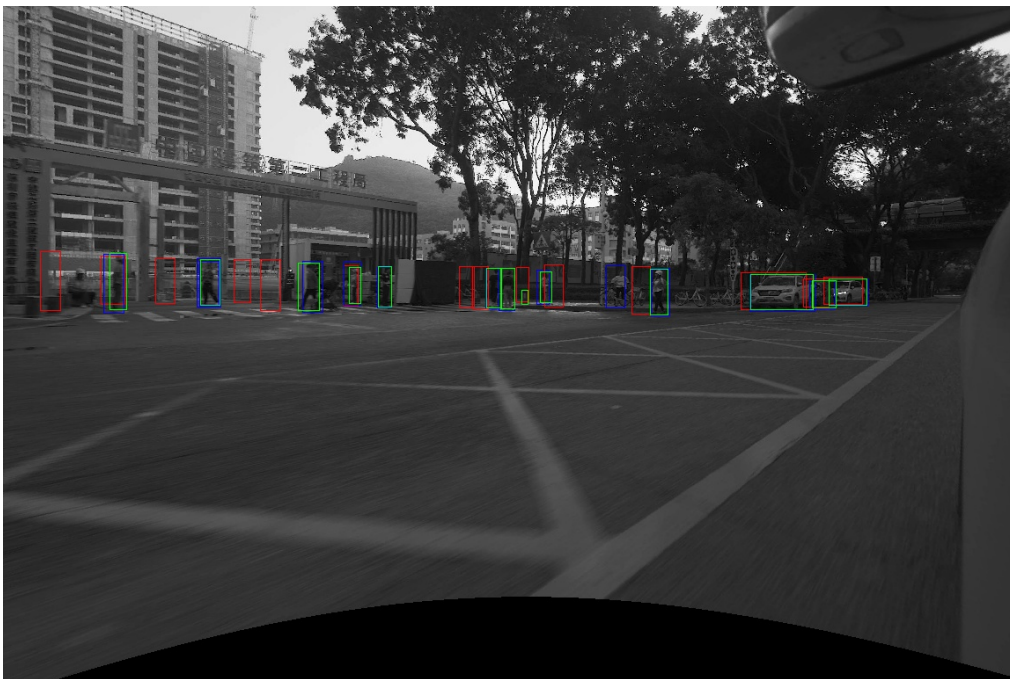
7. 截断位置距离误差过大不关联

- $pv\_post$ 优化的是目标的相对速度 $relative\_vel$ ，所以需要结合自车速度对目标在当前帧的2D框进行预测，

$odom$ 的新位置= $odom$ 的旧位置+绝对速度\*时间

当前自车下的位置= $T*odom$ 的新位置= $T*odom$ 的旧位置+ $T*绝对速度*时间=T*odom$ 的旧位置+自车下绝对速度\*时间

- 使用以上方式将目标预测到当前帧自车下的位置后，投影到当前图像上作为中心点的预测位置，再结合上一帧的尺寸得到当前帧的预测框
- 红色为上一帧观测，蓝色为预测、绿色为当前帧观测



- 使用预测box的IOU作为匹配分数

### 3.3.4.2 匹配

#### 1. 预匹配

- 不断取出评分中匹配分数最大的，当评分大于阈值时记录前后帧平均y值
- 将市场中心较大目标的y值保持较大权重，进行加权平均作为**y方向补偿量**

#### 2. 使用z补偿再进行一次贪心匹配，记录目标对

- #### 3. 最后对未匹配的小目标再进行一次**一维关联**，这个原因是图像视野边缘的目标少量的距离误差，预测中都会产生巨大的横向误差（但整体纵向误差较小），则再使用纵向的y方向交集长度+3D位置进行一次贪心匹配



#### 3.3.4.3 更新

1. 朝向更新 将track朝向翻转为和观测朝向同向
2. 历史信息更新，记录观测信息，记录motion\_type观测信息，同时压入当前观测的roi\_feature
3. 对超过3帧无观测的目标删除，需要预测的目标进行一次2D预测（实际主要是为了light\_feature插入无效信息以及be\_predicted打标）
4. 无历史关联时生成目标
  - 根据mono3d输出设置基础属性，包括深度、id、速度等
  - 初始化卡尔曼滤波器：

base下优化深度kalman\_extend\_ptr

odom下优化位置kalman\_extend\_odom\_ptr

静态目标优化kalman\_extend\_motionless\_ptr

- 根据传感器信息和观测信息设置一些参数信息：

lane\_limite： 需要进行scale计算的横向约束，横向太远目标scale计算失准

near\_distance\_limite： 需要进行scalse计算的纵向约束，太近的目标精度较高不需要scale计算

hide\_frame： pub压帧帧数

motionless\_type\_confidence： 静态目标的置信度信息

confidence\_thresh：静态目标置信度上限，后续根据观测情况发生变化

### 3.3.5 灯信息推理GetLightInfo

- 二阶段网络I/O

input name：'feature'

Input shape: 100\*10\*64

Output name: ['roi\_brake\_light', 'roi\_third\_brake\_light', 'roi\_turn\_light']

Output shape: [(100, 4),(100, 2),(100, 6)]

- 输出字段：brake\_light 可见置信度 亮灯置信度 低位转高位置置信度 高位转低位置置信度（后两个先不用）

roi\_third\_brake\_light 高位刹车灯可见置信度 亮灯置信度（与brake\_light 为或的关系）

roi\_turn\_light（相机视角下）左转可见 左转亮灯 右转可见 右转亮灯 侧向可见 侧向亮灯

- 输出格式：

|              |   |       |   |              |   |   |                                               |
|--------------|---|-------|---|--------------|---|---|-----------------------------------------------|
| signal_light | - | uint8 | 1 | {0, 1, 2, 3} | 0 | - | 目标信号灯：<br>0：都不亮<br>1：左转向灯亮<br>2：右转向灯亮<br>3：双闪 |
| brake_light  | - | uint8 | 1 | {0, 1}       | 0 | - | 目标刹车灯<br>0：没亮<br>1：亮                          |

- 节点网络输入：二阶段网络输入，每个traking\_object里都存储了一个包含十个对象的队列 light\_feature，每个对象都是一个1\*64的向量，按行把每个目标的行向量拷贝到输入内存中，如果不存在目标就用一个全0矩阵填充
- 节点网络输出，把每个输出对应的内存指针存放到一个map里，outputs\_。然后把每个输出的指针都转换成float\*构成float数组，取每个目标对应内存头指针传入NnLightPostProc。
- 刹车灯解析比较简单，目前仅使用高位刹车灯roi\_third\_brake\_light，brake\_lighth\_status uint8 默认为0，1为刹车灯亮
- 转向灯与PV\_NN中开车门一样，网络直出的是在相机视角下的左右，根据朝向和锚点恢复theta，再根据theta对灯和目标的方向绑定。

当目标和观测视线角度较小，也就是基本是车尾正对相机时，左右关系和网络一致，直接输出

当目标和观测视线角度很大，也就是基本车头正对时，左右关系和网络相反，需要反向输出

当目标和观测视线角度接近90°时，说明车身正对自车，需要根据角度正负判断是左侧正对还是右侧正对，使用side\_light取代左右转向灯

- 转向灯受到光照影响大（黄色为主）所以需要比较重的压帧，当某个灯可见数小于3帧时，不信任其网络结果

### 3.3.6 关联radar(不维护)

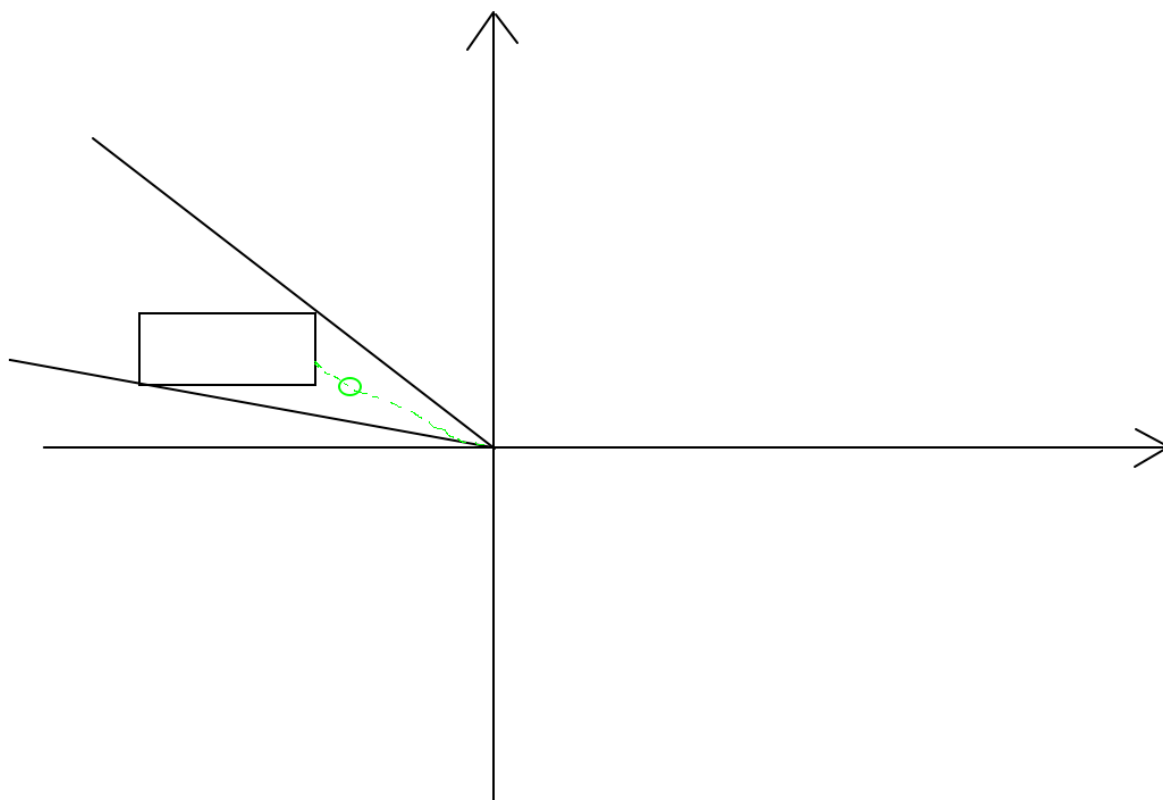
#### 1. radar后处理滤波

- 根据原始信息state中的counter信息，判断是否存在60A丢帧导致的目标乱序，如果乱序则丢弃该消息
- 根据ID信息，在已开辟好的空间中更新目标自车坐标系信息，根据标志位进行生存周期管理（2或者丢失：+1，3：-1，其他：置零），判断位置是否突变，如果存在突变生存周期置0
- Pub 目标（x距离40m，y距离2m内的CIPV直接pub，生存周期小于5不pub，x速度小于5，y速度大于5不pub）

#### 2. pv\_post关联

- 仅对当前帧有观测、朝向和自车夹角小于 $45^\circ$ 、前向距离大于70m的交通工具目标进行关联
- 根据目标框的四个角点计算关联目标的视场角范围，筛选中心点在视场范围内置信度大于0.75的非预测radar目标
- 删除x方向绝对速度差值大于0.1倍距离的radar目标，删除中点差值大于0.1倍距离的radar目标，以bev视角下沿着目标中心点连线的径向距离为主要参考计算评分

打开... 保存 撤销 里微 缩小 100% 放大





- 匈牙利匹配

### 3. pv\_post使用radar

- 将radar中心点投影至目标的可见边上，根据投影点与中心点的位置计算优化**偏移量**
- radar距离观测误差设置为10。因为根据手册radar的绝对误差约为0.5m左右，分辨率为1.7m，考虑到radar只能求得投影点，近距离并没有lidar准确会有一定波动。激光的误差权重设置为0.1倍距离，100m外激光点变少容易关联错误，所以100m外逐渐提高radar的权重。
- radar速度观测误差设置为0.1。根据手册速度精度约为小于0.1m/s。目前mono3d的速度误差权重约为0.1倍距离，10m内radar用于校正波动，10m外基本置信radar
- 对于关联到radar或者lidar后的前5帧，即使没有关联到激光点云也降低状态误差保证观测

### 3.3.7 属性优化

- 为了适应不同场景，存在**静止目标优化**、**有图scale优化**、**无图像优化**、**Y方向优化**和**odom优化**五个优化。**静止目标优化**根据目标的观测高度和世界坐标不变假设进行卡尔曼滤波；**有图scale优化**根据前后帧局部图像的roi尺度变化，使用拓展卡尔曼滤波优化深度方向的位置scale\_estimate\_d和相对速度；**无图优化**则只考虑观测输入优化深度方向的位置和速度；**Y方向**使用加权平均对观测进行平滑；**odom优化**则在odom坐标系下直接优化目标的位置和绝对速度，用于视角非正对、运动不平行自车的目标。

#### 3.3.7.1 静态目标优化

##### 卡尔曼滤波原理

已知目标的参考高度  $h_{ref}$ ，对应相机的内参矩阵为  $K$  或者焦距  $f$  及外参  $[R_{v2s}, t_{v2s}]$ ，静态目标在时间戳  $t_0, \dots, t_{n-1}$  对应有

2D框的最长边观测  $h_0, \dots, h_{n-1}$

静态目标自车坐标系下位置观测  $[x_{v0}, y_{v0}], \dots, [x_{vn-1}, y_{vn-1}]$

自车在世界坐标系下的位姿观测  $[R_{v2w0}, t_{v2w0}], \dots, [R_{v2wn-1}, t_{v2wn-1}]$

将静态目标在世界坐标系的位置  $x_w, y_w$  及真实高度  $h$  作为状态量，即

$$\mathbf{s} = \begin{bmatrix} x_w \\ y_w \\ h \end{bmatrix}$$

采用扩展卡尔曼滤波进行优化，其中状态方程为

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ h \end{bmatrix} + \begin{bmatrix} u_{x_w} \\ u_{y_w} \\ u_h \end{bmatrix} \rightarrow \begin{bmatrix} x_w \\ y_w \\ h \end{bmatrix}$$

其状态方程的噪声协方差矩阵为  $Q$ ，最开始将  $h$  设置成  $h_{ref}$

- 观测为当前帧的base坐标和2D框的高度h

- 位置观测值  $\mathbf{x}_{pi} = \begin{bmatrix} x_{vi} \\ y_{vi} \end{bmatrix}$ ，其观测方程为

$$\mathbf{x}_{pi} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \mathbf{R}_{v2wi}^T \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \left( \begin{bmatrix} x_w \\ y_w \\ h \end{bmatrix} - \mathbf{t}_{v2wi} \right)$$

如果认为  $\mathbf{R}_{v2wi} = \begin{bmatrix} R_{v2wi00} & R_{v2wi01} & R_{v2wi02} \\ R_{v2wi10} & R_{v2wi11} & R_{v2wi12} \\ R_{v2wi20} & R_{v2wi21} & R_{v2wi22} \end{bmatrix}$ ，则上式也可以写成

$$\mathbf{x}_{pi} = \begin{bmatrix} R_{v2wi00} & R_{v2wi10} & 0 \\ R_{v2wi01} & R_{v2wi11} & 0 \end{bmatrix} \left( \begin{bmatrix} x_w \\ y_w \\ h \end{bmatrix} - \mathbf{t}_{v2wi} \right)$$

其对应的  $\mathbf{H}$  矩阵显然为

$$\mathbf{H} = \begin{bmatrix} R_{v2wi00} & R_{v2wi10} & 0 \\ R_{v2wi01} & R_{v2wi11} & 0 \end{bmatrix}$$

该观测值可以来自于激光，也可以来自于视觉，对应于不同的方差

- 目标2D框高度观测

可以定义  $\mathbf{R}_{w2si} = \mathbf{R}_{v2s} \mathbf{R}_{v2wi}^T$ ，根据相似性原理，z\_si为相机坐标系下的深度，将状态量转换到当前帧的相机坐标并取出z值

$$\begin{aligned} h_i &= \frac{hf}{z_{si}} \\ &= \frac{hf}{\begin{bmatrix} 0 & 0 & 1 \end{bmatrix} \mathbf{R}_{v2s} \mathbf{R}_{v2wi}^T \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \left( \begin{bmatrix} x_w \\ y_w \\ h \end{bmatrix} - \mathbf{t}_{v2wi} \right) + \mathbf{t}_{v2s}(z)} \\ &= \frac{hf}{\begin{bmatrix} R_{w2si20} & R_{w2si21} & 0 \end{bmatrix} \left( \begin{bmatrix} x_w \\ y_w \\ h \end{bmatrix} - \mathbf{t}_{v2wi} \right) + \mathbf{t}_{v2s}(z)} \end{aligned}$$

其对应的  $\mathbf{H}$  矩阵为

$$\begin{aligned} \mathbf{H} &= \begin{bmatrix} \frac{\partial h_i}{\partial x_w} & \frac{\partial h_i}{\partial y_w} & \frac{\partial h_i}{\partial h} \end{bmatrix} \\ &= \frac{f}{\left[ \begin{bmatrix} R_{w2si20} & R_{w2si21} & 0 \end{bmatrix} \left( \begin{bmatrix} x_w \\ y_w \\ h \end{bmatrix} - \mathbf{t}_{v2wi} \right) + \mathbf{t}_{v2s}(z) \right]^2} \begin{bmatrix} -hR_{w2si20} & -hR_{w2si21} & [R_{w2si20} & R_{w2si21} & 0] \left( \begin{bmatrix} x_w \\ y_w \\ h \end{bmatrix} - \mathbf{t}_{v2wi} \right) + \mathbf{t}_{v2s}(z) \end{bmatrix} \end{aligned}$$

最终总结扩展卡尔曼滤波的具体步骤为：

预测估计量：



$$\mathbf{A}\mathbf{s} \rightarrow \mathbf{s}$$

预测最小 MSE 矩阵:

$$\mathbf{A}^T \mathbf{M} \mathbf{A} + \mathbf{Q} \rightarrow \mathbf{M}$$

增益矩阵:

$$\mathbf{M} \mathbf{H}^T (\mathbf{H} \mathbf{M} \mathbf{H}^T + \mathbf{C})^{-1} \rightarrow \mathbf{K}$$

修正估计量:

$$\mathbf{s} + \mathbf{K}(\mathbf{x} - h(\mathbf{s})) \rightarrow \mathbf{s}$$

$$\begin{bmatrix} x_w \\ y_w \\ h \end{bmatrix} + \mathbf{K} \left( \begin{bmatrix} x_{vi} \\ y_{vi} \\ h_i \end{bmatrix} - \frac{\begin{bmatrix} R_{v2wi00} & R_{v2wi10} & 0 \end{bmatrix} \begin{pmatrix} \begin{bmatrix} x_w \\ y_w \\ h \end{bmatrix} - \mathbf{t}_{v2wi} \end{pmatrix}}{\begin{bmatrix} R_{v2wi01} & R_{v2wi11} & 0 \end{bmatrix} \begin{pmatrix} \begin{bmatrix} x_w \\ y_w \\ h \end{bmatrix} - \mathbf{t}_{v2wi} \end{pmatrix}} - \frac{hf}{\begin{bmatrix} R_{w2si20} & R_{w2si21} & 0 \end{bmatrix} \begin{pmatrix} \begin{bmatrix} x_w \\ y_w \\ h \end{bmatrix} - \mathbf{t}_{v2wi} \end{pmatrix} + \mathbf{t}_{v2s}(z)} \right) \rightarrow \begin{bmatrix} x_w \\ y_w \\ h \end{bmatrix}$$

修正最小 MSE 矩阵:

$$(\mathbf{I} - \mathbf{K} \mathbf{H}) \mathbf{M} \rightarrow \mathbf{M}$$

其中

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{Q} = \begin{bmatrix} \sigma_{dn} & \\ & \sigma_{vn} \end{bmatrix}$$

$$\mathbf{H} = \begin{bmatrix} \frac{R_{v2wi00}}{fhR_{w2si20}} & \frac{R_{v2wi10}}{fhR_{w2si21}} & 0 \\ \frac{R_{v2wi01}}{fhR_{w2si20}} & \frac{R_{v2wi11}}{fhR_{w2si21}} & 0 \\ \frac{R_{v2wi00}}{fhR_{w2si20}} & \frac{R_{v2wi10}}{fhR_{w2si21}} & 0 \end{bmatrix} \begin{bmatrix} \begin{pmatrix} \begin{bmatrix} x_w \\ y_w \\ h \end{bmatrix} - \mathbf{t}_{v2wi} \end{pmatrix} \\ \begin{pmatrix} \begin{bmatrix} x_w \\ y_w \\ h \end{bmatrix} - \mathbf{t}_{v2wi} \end{pmatrix} \\ \begin{pmatrix} \begin{bmatrix} x_w \\ y_w \\ h \end{bmatrix} - \mathbf{t}_{v2wi} \end{pmatrix} \end{bmatrix} + \mathbf{t}_{v2s}(z)$$

$$\mathbf{C} = \begin{bmatrix} \alpha_1 & & \\ & \beta_1 & \\ & & \gamma_1 \end{bmatrix}$$

## 置信度管理

使用motionless\_type\_confidence控制生存分数，0.3以下为低，0.3-0.7为中，0.7以上为高，confidence\_thresh控制上限。

无激光观测时，如果是预测目标则每帧降低0.1；

有高反点云关联，每帧+0.3分，1帧跳中，3帧跳高

有低反点云关联时，每帧+0.05分，允许跳到中置信度，并且保证如果有高反点一帧到高

纯视觉，每帧+0.05分，前向30m内突然出现的不允许跳中

### 3.3.7.1 scale计算（红粗表示魔鬼参数）

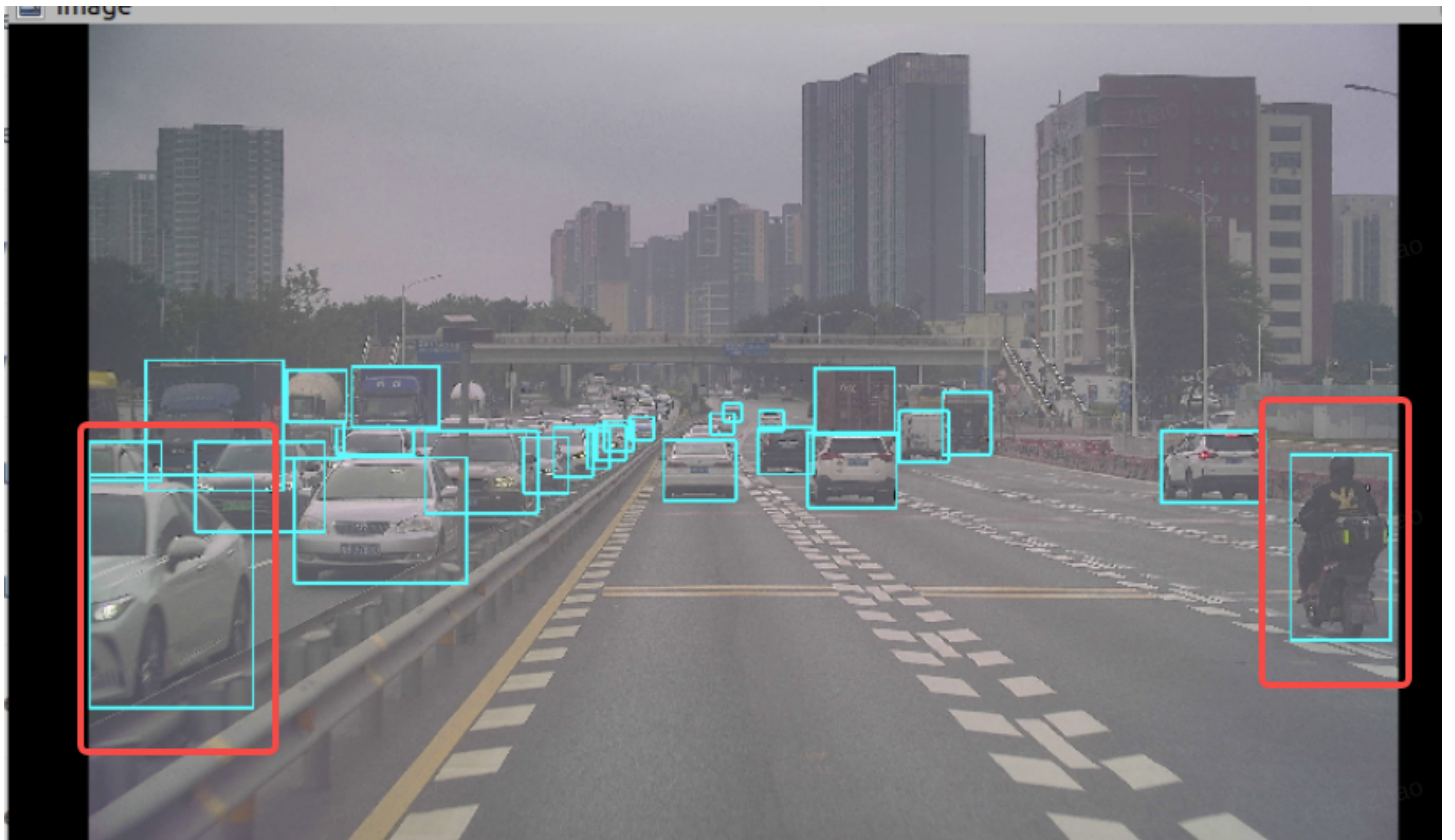
- 非车辆目标、预测目标、遮挡目标、图像无效目标、侧前目标、非图像中央目标不使用scale计算

#### 1. 遮挡图像预处理

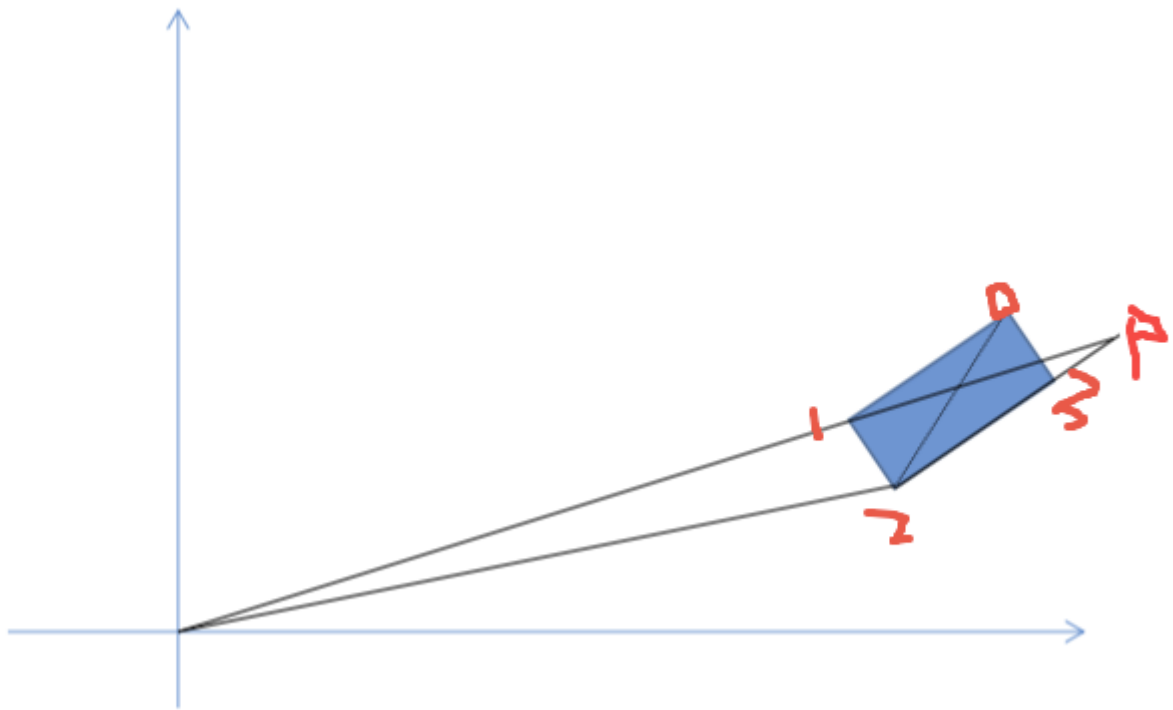
- 遍历当前帧目标列表，提取可见框和脑补完整框
- 如果脑补完整框面积大于**2倍**可见框面积，标记该目标为强遮挡
- 利用矩形求交确定两两目标可见框重叠范围，建立遮挡区域mask矩阵，将遮挡区域的mask设置为0

#### 2. 侧向目标正对图像提取

- 如果可见框左侧边缘小于**5**，或者右侧边缘大于图像宽度**-5**，认为是边缘超车目标，返回false，不处理



- （根据3D框提取**车尾图像**占比，以逆时针分布的角点，车尾区域占比就是从原点到1号点的连线对对角线切分的比例，根据相似性，对对角线的切分就等于1-0线段长度和2-p线段长度的比值，知道比值后可以根据截断方向将车尾图像从原图中截取出来



### 3. scale计算

- 将输入的两幅相邻图像resize相同倍数，到最小边长等于64，并对小图做直方图均衡提高对比度
- GradCal利用sobel算子计算 $x$   $y$ 两个方向的梯度信息，以及每个点的梯度幅值和梯度夹角
- 利用输入的重叠mask对 $gx$   $gy$   $mag$   $angle$ 四个矩阵进行去重叠处理
- TemplateMatch对于上一帧梯度图进行一定裁剪留出偏移余量后利用**模板匹配**预配准得到一个初步的横纵offset
- EdgePreprocess利用非极大值抑制方法，根据横纵梯度和总梯度，提取亚像素级的**横纵方向**强边缘特征点和对应的梯度方向

- 对于src图像中的每个边缘点，在5\*5的邻域内加上步骤2求出的偏移量，作为target图像中候选点坐标，最近邻插值候选点坐标得到真实的像素点，判断该像素点是否是有效边缘，并判断该点梯度方向是否与待匹配点相同，如果匹配到了有效边缘点，对匹配点在梯度方向上进行插值，得到亚像素边缘

求解该最小二乘问题来得到帧间的尺度变换和平移量（模板匹配后认为t为0）

$$s, t = \arg \min \sum_{i=0}^n \|n_{q_i}^T (sp_i + t - q_i)\|^2$$

使用了梯度方向近似垂直坐标轴假设（nx\*ny≈0）实际计算方程如下：

上式可以展开得到

$$f(s, t) = \sum_{i=0}^n \|n_{q_i}^T (sp_i + t - q_i)\|^2 = \sum_{i=0}^n (sp_i - q_i)^T n_{q_i} n_{q_i}^T (sp_i - q_i) + t^T n_{q_i} n_{q_i}^T t + 2(sp_i - q_i)^T n_{q_i} n_{q_i}^T t$$

利用  $\frac{\partial f(s, t)}{\partial t} = 0$

$$\begin{aligned} \frac{\partial f(s, t)}{\partial t} &= \sum_{i=0}^n 2n_{q_i} n_{q_i}^T t + 2n_{q_i} n_{q_i}^T (sp_i - q_i) = 0 \\ \Rightarrow t &= \left( \sum_{i=0}^n n_{q_i} n_{q_i}^T \right)^{-1} \sum_{i=0}^n n_{q_i} n_{q_i}^T (q_i - sp_i) \end{aligned}$$

求解可以得到

$$\begin{aligned} f(s, t) &= \sum_{i=0}^n (sp_i - q_i)^T n_{q_i} n_{q_i}^T (sp_i - q_i) + t^T n_{q_i} n_{q_i}^T t + 2(sp_i - q_i)^T n_{q_i} n_{q_i}^T t \\ &= \sum_{i=0}^n (sp_i - q_i)^T n_{q_i} n_{q_i}^T (sp_i - q_i) - \sum_{i=0}^n (sp_i - q_i)^T n_{q_i} n_{q_i}^T \left( \sum_{i=0}^n n_{q_i} n_{q_i}^T \right)^{-1} \sum_{i=0}^n n_{q_i} n_{q_i}^T (sp_i - q_i) \end{aligned}$$

继续对 s 求导可以得到

$$\begin{aligned} \frac{\partial f(s, t)}{\partial s} &= 0 \\ \Rightarrow 2s \sum_{i=0}^n p_i^T n_{q_i} n_{q_i}^T p_i - 2 \sum_{i=0}^n p_i^T n_{q_i} n_{q_i}^T q_i \\ &\quad - 2s \sum_{i=0}^n p_i^T n_{q_i} n_{q_i}^T \left( \sum_{i=0}^n n_{q_i} n_{q_i}^T \right)^{-1} \sum_{i=0}^n n_{q_i} n_{q_i}^T p_i + 2 \sum_{i=0}^n p_i^T n_{q_i} n_{q_i}^T \left( \sum_{i=0}^n n_{q_i} n_{q_i}^T \right)^{-1} \sum_{i=0}^n n_{q_i} n_{q_i}^T q_i = \\ \Rightarrow s &= \frac{\sum_{i=0}^n p_i^T n_{q_i} n_{q_i}^T q_i - \sum_{i=0}^n p_i^T n_{q_i} n_{q_i}^T \left( \sum_{i=0}^n n_{q_i} n_{q_i}^T \right)^{-1} \sum_{i=0}^n n_{q_i} n_{q_i}^T q_i}{\sum_{i=0}^n p_i^T n_{q_i} n_{q_i}^T p_i - \sum_{i=0}^n p_i^T n_{q_i} n_{q_i}^T \left( \sum_{i=0}^n n_{q_i} n_{q_i}^T \right)^{-1} \sum_{i=0}^n n_{q_i} n_{q_i}^T p_i} \end{aligned}$$

令

$$s = \frac{A - B * C * D}{E - B * C * F}$$

注意到  $\mathbf{n} = \begin{bmatrix} n_x \\ n_y \end{bmatrix}$  是二维上的向量，因此

$$\left( \sum \mathbf{n} \mathbf{n}^T \right)^{-1} = \left[ \begin{array}{cc} \sum n_x^2 & \sum n_x n_y \\ \sum n_x n_y & \sum n_y^2 \end{array} \right]^{-1} = \frac{1}{\sum n_x^2 \sum n_y^2 - (\sum n_x n_y)^2} \left[ \begin{array}{cc} \sum n_y^2 & -\sum n_x n_y \\ -\sum n_x n_y & \sum n_x^2 \end{array} \right]$$

如果拆成两个方向上联合求解，即意味着认为

$$\mathbf{n} \mathbf{n}^T = \begin{bmatrix} n_x^2 & n_x n_y \\ n_x n_y & n_y^2 \end{bmatrix} \approx \begin{bmatrix} n_x^2 & 0 \\ 0 & n_y^2 \end{bmatrix}$$

因此有

$$\mathbf{t} = \begin{bmatrix} t_x \\ t_y \end{bmatrix} = \begin{bmatrix} \frac{\sum_{i=0}^{n_x} n_{q_{ix}}^2 (q_{ix} - s p_{ix})}{\sum_{i=0}^{n_x} n_{q_{ix}}^2} \\ \frac{\sum_{i=0}^{n_y} n_{q_{iy}}^2 (q_{iy} - s p_{iy})}{\sum_{i=0}^{n_y} n_{q_{iy}}^2} \end{bmatrix}$$

$$s = \frac{\sum_{i=0}^{n_x} n_{q_{ix}}^2 p_{ix} q_{ix} - \frac{\sum_{i=0}^{n_x} n_{q_{ix}}^2 p_{ix} \sum_{i=0}^{n_x} n_{q_{ix}}^2 q_{ix}}{\sum_{i=0}^{n_x} n_{q_{ix}}^2} + \sum_{i=0}^{n_y} n_{q_{iy}}^2 p_{iy} q_{iy} - \frac{\sum_{i=0}^{n_y} n_{q_{iy}}^2 p_{iy} \sum_{i=0}^{n_y} n_{q_{iy}}^2 q_{iy}}{\sum_{i=0}^{n_y} n_{q_{iy}}^2}}{\sum_{i=0}^{n_x} n_{q_{ix}}^2 p_{ix}^2 - \frac{(\sum_{i=0}^{n_x} n_{q_{ix}}^2 p_{ix})^2}{\sum_{i=0}^{n_x} n_{q_{ix}}^2} + \sum_{i=0}^{n_y} n_{q_{iy}}^2 p_{iy}^2 - \frac{(\sum_{i=0}^{n_y} n_{q_{iy}}^2 p_{iy})^2}{\sum_{i=0}^{n_y} n_{q_{iy}}^2}}$$

其中  $n_x + n_y = n$   $n_x$  表示沿着  $y$  方向(列方向)的边缘， $n_y$  表示沿着  $x$  方向(行方向)的边缘

$$A = \sum_{i=0}^{n_x} n_{q_{ix}}^2 p_{ix} q_{ix}$$

$$B = \sum_{i=0}^{n_x} n_{q_{ix}}^2 p_{ix}$$

$$C = \sum_{i=0}^{n_x} n_{q_{ix}}^2 q_{ix}$$

$$D = \sum_{i=0}^{n_x} n_{q_{ix}}^2$$

$$E = \sum_{i=0}^{n_x} n_{q_{ix}}^2 p_{ix}^2$$

可以将s的写作:

$$s = \frac{A_x - B_x * C_x / D_x + A_y - B_y * C_y / D_y}{E_x - B_x^2 / D_x + E_y - B_y^2 / D_y}$$

如果源图像是前一帧图像，目标图像是当前帧，则有  $\frac{d_i}{d_{i-1}} = \frac{1}{s_i}$

### 3.3.7.1 卡尔曼滤波器参数

#### 卡尔曼滤波器原理

采用扩展卡尔曼滤波进行优化，其中状态方程为

$$\begin{bmatrix} d_n \\ v_n \end{bmatrix} = \begin{bmatrix} 1 & t_n - t_{n-1} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} d_{n-1} \\ v_{n-1} \end{bmatrix} + \begin{bmatrix} u_d \\ u_v \end{bmatrix}$$

其状态方程的噪声协方差矩阵为  $\mathbf{Q}_n$

如果存在直接对于距离的观测值  $d_o$  和速度的观测值  $s_o$ ，其观测方程为

$$\begin{bmatrix} d_{o1} \\ \vdots \\ d_{op} \\ v_{o1} \\ \vdots \\ v_{oq} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ \vdots & \vdots \\ 1 & 0 \\ 0 & 1 \\ \vdots & \vdots \\ 0 & 1 \end{bmatrix} \begin{bmatrix} d_n \\ v_n \end{bmatrix} + \begin{bmatrix} w_{\alpha_1} \\ \vdots \\ w_{\alpha_p} \\ w_{\beta_1} \\ \vdots \\ w_{\beta_q} \end{bmatrix}$$

其观测噪声的协方差矩阵为  $\mathbf{C}_n$

这些直接观测值可以来源于传感器，也可以来源于神经网络的结果，对于传感器得到的观测值，可以根据传感器的精度设定噪声的方差，对于神经网络给出的观测值，可以根据经验给出噪声的方差（激光距离噪声  $\alpha_{lidar} = 0.1$ ，毫米波雷达距离噪声  $\alpha_{radar} = 0.1$ ，视觉距离噪声  $\alpha_{camera} = d_n$ ，毫米波雷达速度噪声  $\beta_{radar} = 0.1$ ，视觉速度噪声  $\beta_{camera} = 0.1d_n$ ）

除此之外还有 scale 的约束，假设当前帧  $n$  和窗口内  $j$  帧进行了 scale 计算，利用匀速运动假设，则有

$$s_{jn} = \frac{l_n}{l_j} = \frac{d_j}{d_n} = \frac{d_n - (t_n - t_j)v_n}{d_n} = 1 - \frac{v_n}{d_n}(t_n - t_j) = h_j(d_n, v_n)$$

该方程为非线性方程，可以利用扩展卡尔曼滤波进行求解，这里求解对应  $s$  部分的  $\mathbf{H}_n$  矩阵

$$\mathbf{H}_n = \begin{bmatrix} \frac{\partial s_{jn}}{\partial d_n} & \frac{\partial s_{jn}}{\partial v_n} \end{bmatrix} \bigg|_{d_n=\hat{d}_n, v_n=\hat{v}_n} = \begin{bmatrix} \frac{\hat{v}_n}{\hat{d}_n^2}(t_n - t_j) & -\frac{1}{\hat{d}_n}(t_n - t_j) \end{bmatrix}$$

如果计算了多个 scale，均可以添加进入观测，最终总结扩展卡尔曼滤波的具体步骤为：

预测估计量：

$$\begin{bmatrix} \hat{d}_n \\ \hat{v}_n \end{bmatrix} = \mathbf{A}_n \begin{bmatrix} d_{n-1} \\ v_{n-1} \end{bmatrix}$$

预测最小 MSE 矩阵：

$$\hat{\mathbf{M}}_n = \mathbf{A}_n^T \mathbf{M}_{n-1} \mathbf{A}_n + \mathbf{Q}_n$$

增益矩阵：

$$\mathbf{K}_n = \hat{\mathbf{M}}_n \mathbf{H}_n^T (\mathbf{H}_n \hat{\mathbf{M}}_n \mathbf{H}_n^T + \mathbf{C}_n)^{-1}$$

修正估计量：

$$\begin{bmatrix} d_n \\ v_n \end{bmatrix} = \begin{bmatrix} \hat{d}_n \\ \hat{v}_n \end{bmatrix} + \mathbf{K}_n \left( \begin{bmatrix} d_{o1} \\ \vdots \\ d_{op} \\ v_{o1} \\ \vdots \\ v_{oq} \\ s_{j_1 n} \\ \vdots \\ s_{j_r n} \end{bmatrix} - \begin{bmatrix} \hat{d}_n \\ \vdots \\ \hat{d}_n \\ \hat{v}_n \\ \vdots \\ \hat{v}_n \\ 1 - \frac{\hat{v}_n}{\hat{d}_n} (t_n - t_{j_1}) \\ \vdots \\ 1 - \frac{\hat{v}_n}{\hat{d}_n} (t_n - t_{j_r}) \end{bmatrix} \right)$$

修正最小 MSE 矩阵：

$$\mathbf{M}_n = (\mathbf{I} - \mathbf{K}_n \mathbf{H}_n) \hat{\mathbf{M}}_n$$

其中

$$\mathbf{A}_n = \begin{bmatrix} 1 & t_n - t_{n-1} \\ 0 & 1 \end{bmatrix}, \quad \mathbf{Q} = \begin{bmatrix} \sigma_{dn} & \\ & \sigma_{vn} \end{bmatrix}$$

$$\mathbf{H}_n = \begin{bmatrix} 1 & 0 \\ \vdots & \vdots \\ 1 & 0 \\ 0 & 1 \\ \vdots & \vdots \\ 0 & 1 \\ \frac{\hat{v}_n}{\hat{d}_n^2} (t_n - t_{j_1}) & -\frac{1}{\hat{d}_n} (t_n - t_{j_1}) \\ \vdots & \vdots \\ \frac{\hat{v}_n}{\hat{d}_n^2} (t_n - t_{j_r}) & -\frac{1}{\hat{d}_n} (t_n - t_{j_r}) \end{bmatrix}, \quad \mathbf{C}_n = \begin{bmatrix} \alpha_1 & & & & & \\ & \ddots & & & & \\ & & \alpha_p & & & \\ & & & \beta_1 & & \\ & & & & \ddots & \\ & & & & & \beta_q \\ & & & & & & \gamma_1 & \ddots \\ & & & & & & & \ddots & \gamma_r \end{bmatrix}$$

- 最后卡尔曼滤波输出时，还会根据窗口内速度变化估计加速度，根据加速度进行进一步窗口平滑

### 滤波器初始化

- d和v初始化为网络**输入距离值**和**0**
- M 设置为0.1倍距离，初始化时越远越相信观测，A为单位阵
- 状态误差矩阵Q设置为  $\mathbf{Q} = \begin{bmatrix} 1 & \\ & 0.1 \end{bmatrix}$ ，速度尽量平滑

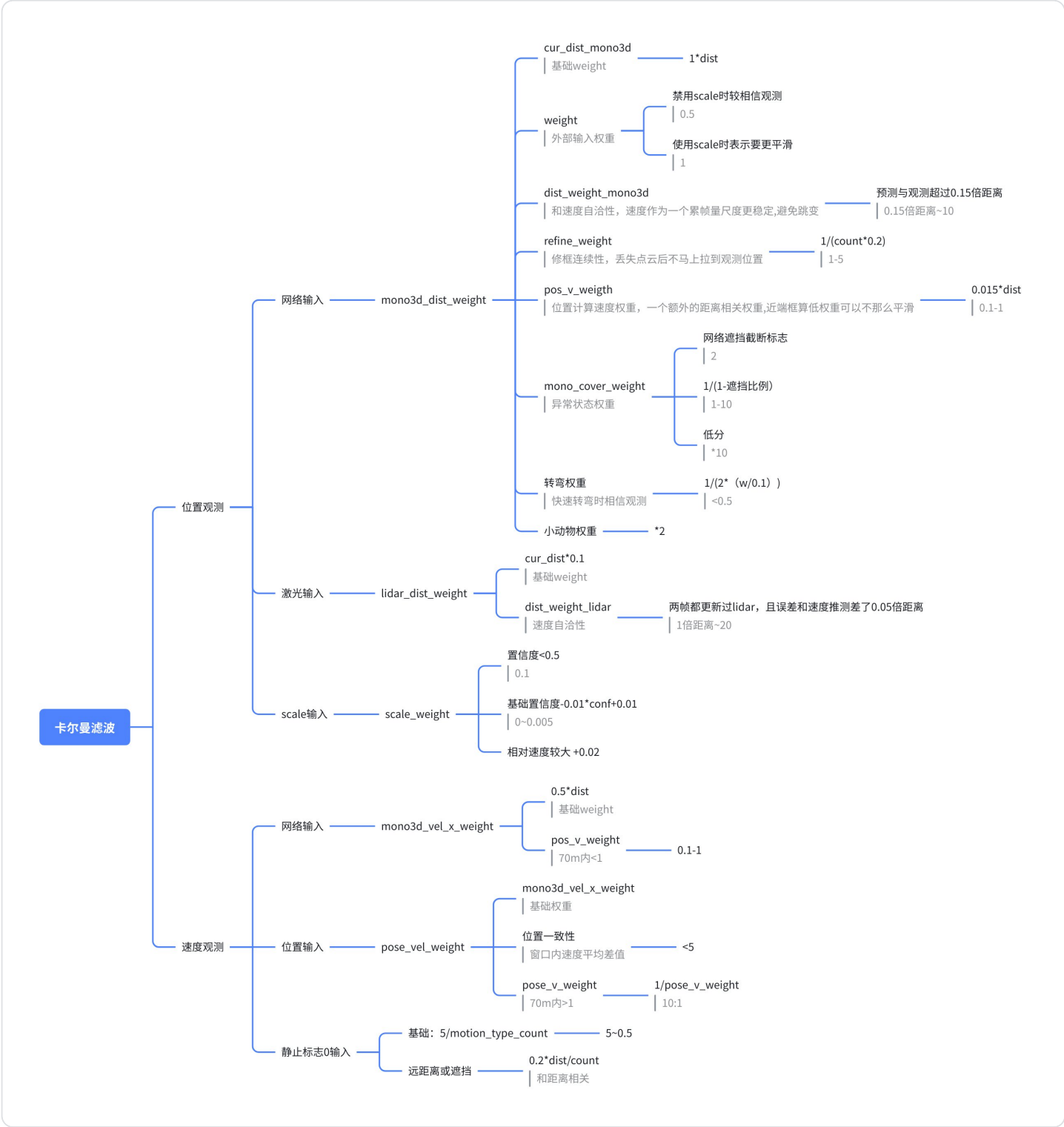
### 3.3.7.2 深度方向属性优化



利用scale进行属性优化

 weight指的是观测误差量级，越大在优化中权重越低

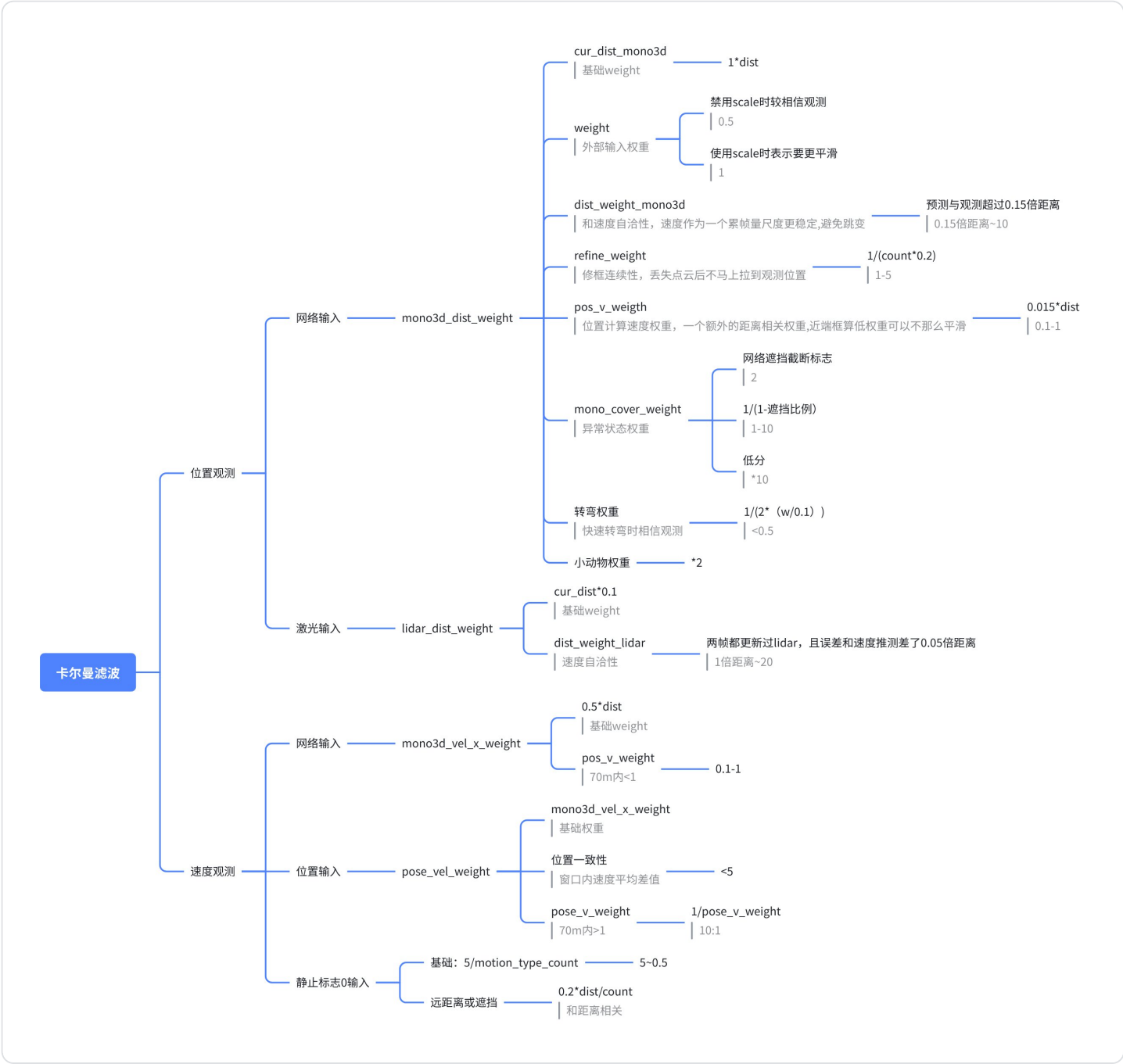
- 观测源一致时，利用优化后的距离计算速度，并进行滑动窗口（窗口尺寸为10）平滑，当窗口内值大于3时，将该速度作为速度输入值。



无图像优化

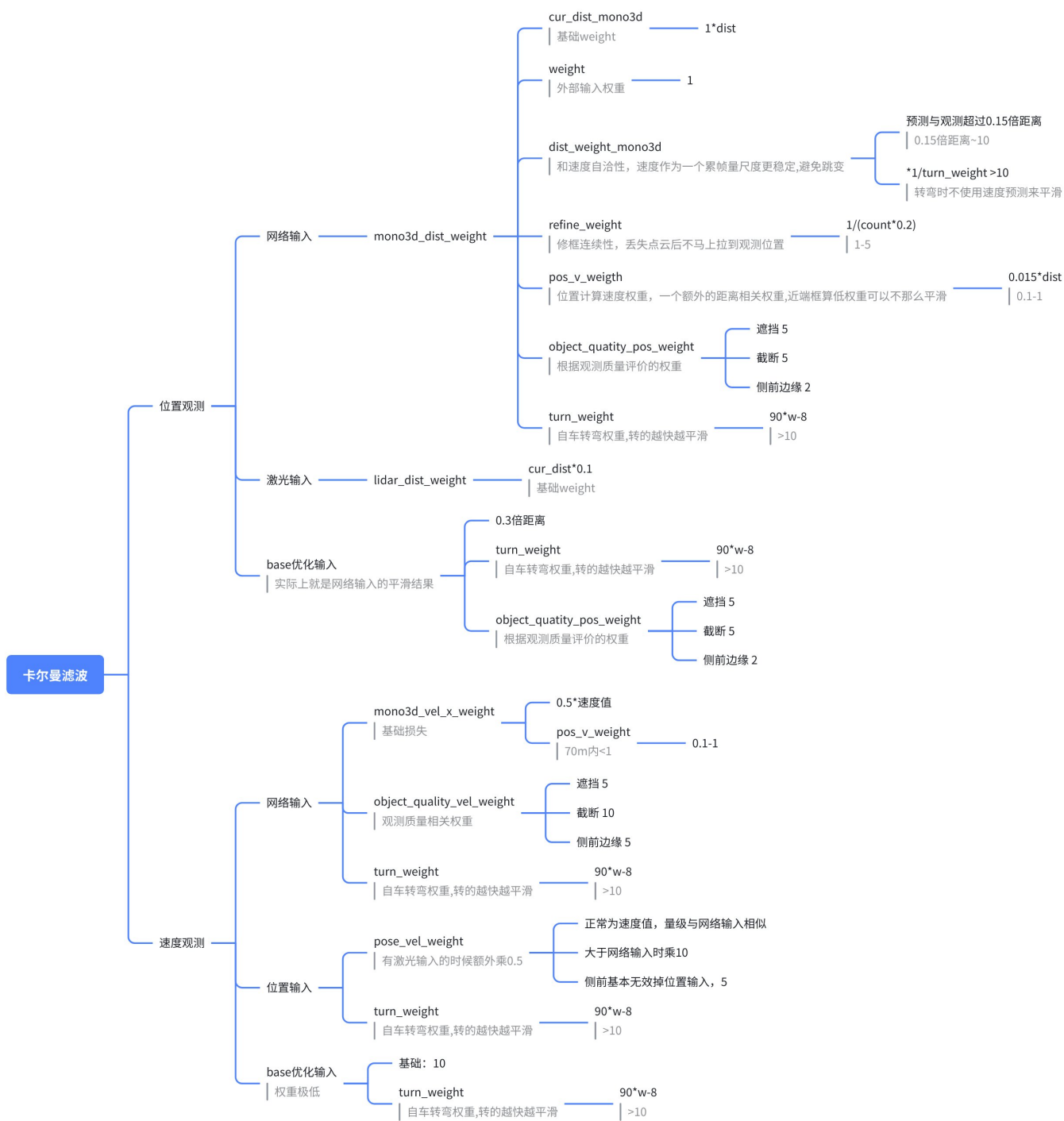
 weight指的是观测误差量级，越大在优化中权重越低

- 仍然使用上方的拓展卡尔曼滤波，只是不计算scale，仅输入距离网络输入和平滑后速度



### ODOM优化

- 实验中发现，当目标或者自车高速转弯、目标受到严重遮挡、无论是网络还是scale计算都会很不稳定，特别是无scale约束时，纯网络会出现严重的静止目标偏移，而odom优化可以通过横纵平滑优化这个问题
- odom优化的目标是尽量保持平滑，优化模型就是上方的d优化改为x\_odom y\_odom
- 输入除了网络输入、雷达输入，也可以包含车体优化的输出



## 其他属性优化

- Y方向速度及距离，Y方向速度利用Y方向位移两帧之间计算速度，同时和网络输入取小值进行5帧平滑
- Y方向距离，如果X方向距离小于**5m**直接使用网络值，否则按照优化得到的X值和网络输入的X/Y比例，等比例计算估计值，5m内XY比值较小，小幅度的X差值也容易导致比较大的偏移，而近距离网络Y估计准确度较高
- 对尺寸的XYZ进行**5帧**的滑动窗口平滑

## 3.3.8 输出

- 预测的目标，除非是有点云关联的静目标，否则不pub
- 静目标低置信度压帧2帧再pub

### 3.3.8.1 属性调整

1. 尺寸要做5帧窗口平滑
2. 静目标位置使用odom位置转换得到；
3. 普通动目标x方向距离由scale\_estimate\_d偏置得到；
4. 侧前目标y都由观测直接得到；其他相机近距离目标y由观测得到，剩下的沿着观测射线和scale\_estimate\_d得到
5. 速度由相对速度加上自车速度得到

### 3.3.8.2 is\_box\_refine机制

- 置信度由位置置信度和速度置信度共同构成
1. 速度置信度 can\_be\_trust
    - 压帧过程中(3帧) false
    - 无定位 false
    - 车辆类目标速度朝向和输出朝向角度大于45°，false
    - vru目标速度大于5 false
    - 速度小于0.5的低速目标直接等于true
    - 小动物目标 false
  2. 位置置信度 have\_pos\_refine
    - 有激光或radar校正为true
  3. 输出机制
    - 允许激光输入时：0位置速度均不可信 1表示位置经过点云校正速度不可信 2 位置速度均可新 3 速度可信但位置没有经过点云校正
    - 不允许激光输入时：0表示不可信，1表示速度可信

### 3.3.8.3 nn\_state

- 网络低分的非锥桶目标nn\_state==1
- 遮挡和截断透传内部判断，略严格于网络

### 3.3.8.4 拓展字段输出extended\_key extended\_value

1. 锥桶置信度 PV\_POST\_STATIC\_CONFIDENCE\_KEY 52
  - 根据motionless\_type\_confidence，小于0.3低（2），小于0.7中（1），大于0.7高（0）

2. 锥桶置信度float值 PV\_OBJ\_STATIC\_CONF\_KEY 54 数据挖掘用
3. 关联Radar id PV\_POST\_CORRELATION\_RADAR\_ID\_KEY 53 透传
4. 截断比值 PV\_PROJ\_RATIO\_KEY\_1 PV\_PROJ\_RATIO\_KEY\_2 50 51 透传
5. 灯信息网络分数透传 90-95 数据挖掘用

### 3.3.8.5 box恢复

- 同2.4.3，主要为了尺度统一以及后续应用中标定应用方便

## 3.4 部分debug功能

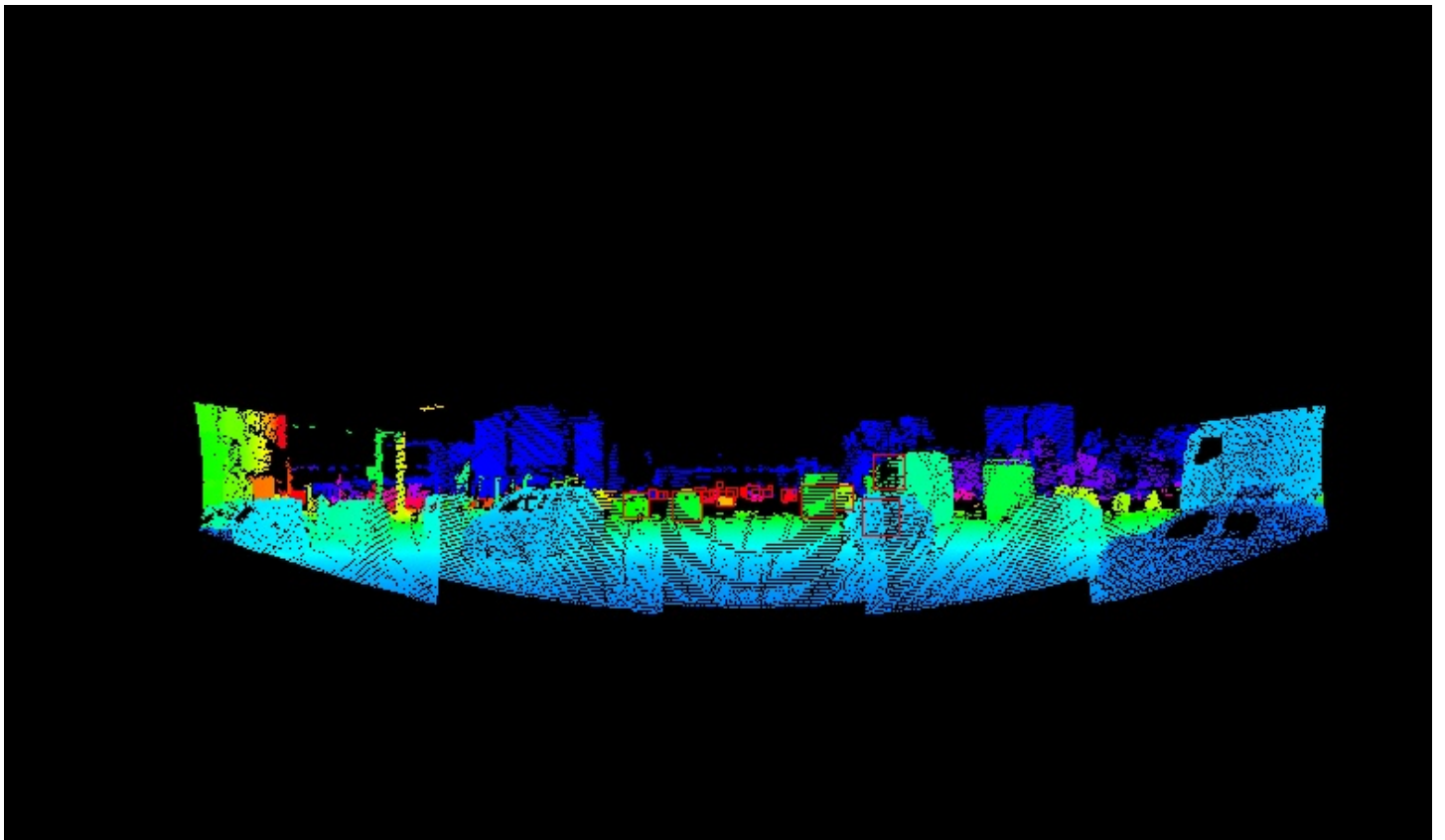
1. 预测图存图 用于debug关联问题 #define SAVE\_PREDICT\_IMAGE

mono3d/algorithm/pv\_post\_process/include/hyper\_vision/perception/pv\_post\_process/multi\_view\_post\_process\_task.hpp

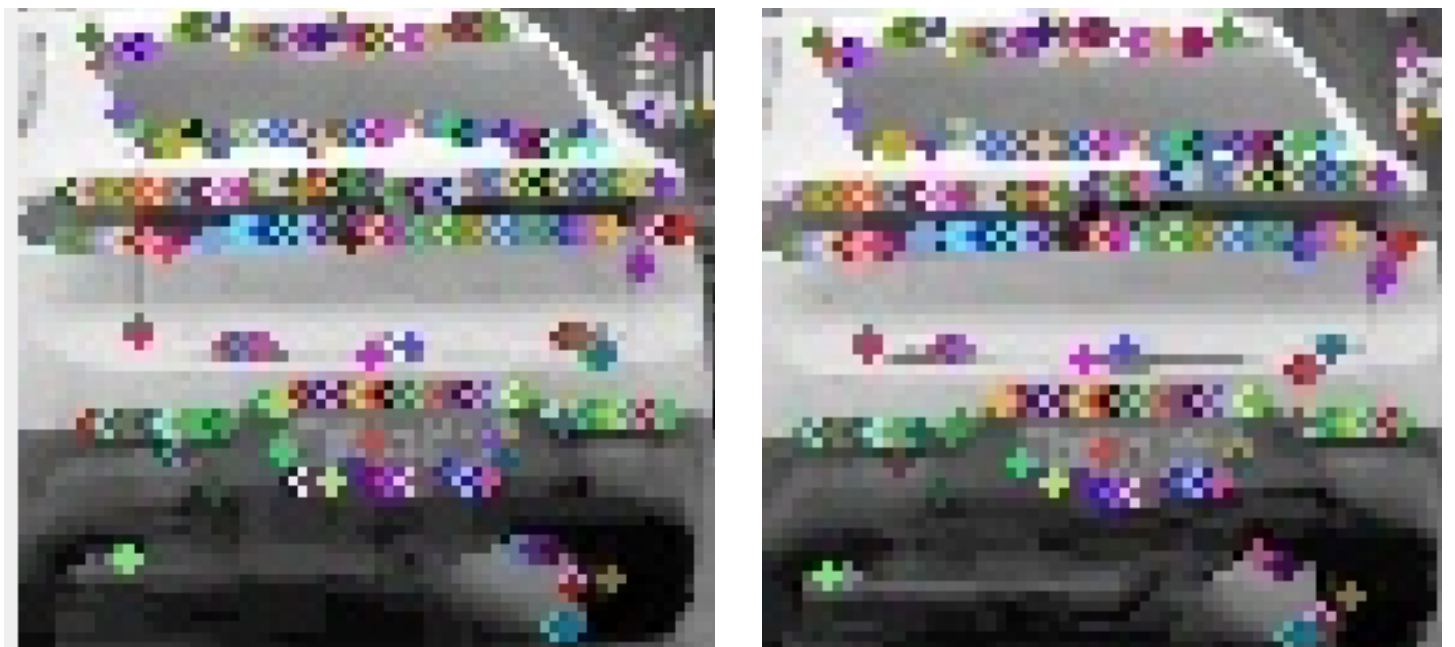


2. 存储rangeimage的dist图和反射率图 #define SAVE\_IMAGE

mono3d/algorithm/pv\_post\_process/src/hyper\_vision/perception/pv\_post\_process/multi\_view\_post\_process\_task.cpp



3. scale匹配点对可视化 bool debug\_mode\_ = true;  
 mono3d/algorithm/pv\_post\_process/include/hyper\_vision/perception/pv\_post\_process/scale\_  
 estimate.hpp



4. 文字信息打印  
 // #define SAVE\_TXT  
 // #define SAVE\_PV\_POST\_INPUT\_INFO  
 // #define PRINT\_LIGHT\_INFO

mono3d/algorithm/pv\_post\_process/include/hyper\_vision/perception/pv\_post\_process/post\_p  
roces\_tracking\_objs.hpp